



**Intelligent Information System Supporting
Observation, Searching and Detection for
Security of Citizens in Urban Environment**



*European Seventh Framework Programme
FP7-218086-Collaborative Project*

D8.3 Specification of new constructed block cipher and evaluation of its vulnerability to errors

The INDECT Consortium

AGH – University of Science and Technology, AGH, Poland
Gdansk University of Technology, GUT, Poland
InnoTec DATA GmbH & Co. KG, INNOTEC, Germany
Grenoble INP (Ensimag), INP, France
MSWiA¹ - General Headquarters of Police (Polish Police), GHP, Poland
Moviquity, MOVIQUITY, Spain
PSI Transcom GmbH, PSI, Germany
Police Service of Northern Ireland, PSNI, United Kingdom
Poznan University of Technology, PUT, Poland
Universidad Carlos III de Madrid, UC3M, Spain
Technical University of Sofia, TU-SOFIA, Bulgaria
University of Wuppertal, BUW, Germany
University of York, UoY, Great Britain
Technical University of Ostrava, VSB, Czech Republic
Technical University of Kosice, TUKE, Slovakia
X-Art Pro Division G.m.b.H., X-art, Austria
Fachhochschule Technikum Wien, FHTW, Austria

¹MSWiA (Ministerstwo Spraw Wewnętrznych i Administracji) – Ministry of Interior Affairs and Administration. Polish Police is dependent on the Ministry

© Copyright 2010, the Members of the INDECT Consortium
Document Information

Contract Number	218086
Deliverable name	<i>Specification of new constructed block cipher and evaluation of its vulnerability to errors</i>
Deliverable number	D8.3
Editor(s)	Marcin Niemiec (AGH) niemiec@kt.agh.edu.pl
Author(s)	Marcin Niemiec (AGH) Łukasz Machowski (AGH) Marcin Świąty (AGH) Jakub Dudek (AGH) Łukasz Romański (AGH) Nikolai Stoianov (TU-SOFIA)
Reviewer(s)	Manuel Urueña (UC3M) Nikolai Stoianov (TUS) Plamen Vichev (TU-SOFIA) – Ethics Board review Cezary Basek (Polish Police) – End-Users' review Katarzyna Wasilewska (Polish Police) – End-Users' review
Dissemination level	Public
Contractual date of delivery	Month 24 (December 2010)
Delivery date	31.12.2010
Status	<ver. 1.0 (03.12.2010)>
Keywords	<i>block cipher, s-box, cipher's vulnerability</i>



This project is funded under 7th Framework Program

Table of Contents

1	Executive Summary	6
2	Introduction	7
3	Requirements	8
4	Cryptography background	9
4.1	Basic functions	9
4.2	Security features	13
4.3	Operation modes	15
5	Contemporary block ciphers	17
5.1	DES	17
5.2	Triple DES	18
5.3	IDEA	18
5.4	AES	18
6	New block cipher	22
6.1	General structure	22
6.2	The idea of ‘basic functions’	24
6.3	New substitution boxes	26
6.4	Keys	29
6.5	The algorithm	29
7	Evaluation of security and vulnerabilities	31
7.1	Cryptanalysis	31
7.2	The simulator of block ciphers	39
7.3	Tested features	43
8	Initial tests of the new cipher	61
9	Ethical issues and security considerations	68
10	Conclusions	69
	Abbreviations	70
	References	71
	Document Updates	72

(This page is left blank intentionally)

1 Executive Summary

INDECT: "Intelligent information system supporting observation, searching and detection for security of citizens in urban environment" is a Collaborative Research Project funded by the EU 7th Framework Program. Its main aim is to develop cost-efficient tools for helping European Police services to enforce the law and guarantee the protection of European citizens. These tools must comply with both country-level laws as well as European-level directives including, among many others, the European Declaration on Human Rights.

This deliverable D8.3 "Specification of new constructed block cipher and evaluation of its vulnerability to errors" presents the design of a new block cipher. This innovative encryption/decryption algorithm is described in detail. Also some tests and simulations have been performed to check and evaluate the security level. The simulation software is also introduced in one of the chapters.

It is worth to mention that the implementation of the new block cipher is presented in the companion deliverable: D9.13 "New block ciphers". The developed application is able to encrypt/decrypt ordinary plaintext as well as any kind of file.

2 Introduction

One of the main research areas in Work Package 8 of INDECT (Intelligent information system supporting observation, searching and detection for security of citizens in urban environment) project is the development of cryptographic algorithms. The deliverable D8.3 “Specification of new constructed block cipher and evaluation of its vulnerability to errors” presents new encryption/decryption algorithm constructed in AGH. The new cipher is presented in this document in detail and has been implemented in C++ object-oriented programming language. The developed application is presented in the companion deliverable D9.13 “New block ciphers”.

This document includes sections dedicated to security and vulnerability of the new cipher. The cryptanalysis of new encryption/decryption algorithm has been performed, as well as some additional security tests. This deliverable also presents new simulator—the application dedicated to evaluation of block cipher. This simulator was jointly developed by AGH, TUS and PUT.

At the beginning of the document we define the most important requirements which should be met by a block cipher. Next chapter introduces cryptography basics such as: crucial transformations, security features or modes of encryption/decryption. We presented also main contemporary block ciphers (DES, Triple DES, IDEA, and AES) which are widely used in practice.

The crucial chapter “New block cipher” presents the idea of this novel solution. In this chapter we described the new algorithm in detail. Next chapter presents an evaluation of its security and vulnerabilities, performed mainly by means of the new simulator of substitution boxes and block ciphers. This chapter also includes considerations about the cryptanalysis of the new cipher. In separated chapter, we present some initial vectors and security tests of these patterns.

3 Requirements

At the beginning of the symmetric block cipher development, we defined the main requirements which should be met by new solution. We have emphasized three of them: high resistance to cryptanalysis, wide key space, and fast data encryption.

- **High robustness to cryptanalysis**

The most important characteristic that any cipher should possess is resistance to two basic cryptanalysis types: linear cryptanalysis and differential cryptanalysis. The algorithm parameters that are directly responsible for such features are presented in this deliverable in detail. In particular, the parameters that are essential for these features are: non-linearity, which hardens the algorithm against linear cryptanalysis, and a good XOR-distribution table that counteracts the existence of highly probable differential characteristics.

- **Wide key space**

The number of possible keys (key space) defines the time that is needed for breaking the algorithm using exhaustive key search (in other words, a *brute-force attack*). It is also relevant that in case of breaking the algorithm (and by that, we mean reducing the searched key space), still the number of the keys left to check should require enormous computational power or/and memory in order to find the key in reasonable time.

- **Fast data encryption**

Encryption rate is directly connected to the number of rounds that the algorithm performs. Too low number of rounds weakens the algorithm, but too much rounds makes the whole algorithm slower. On the other hand, a bigger number of rounds increases the security level of the cipher. It is better when a round contains less transformations but it is processed more times.

4 Cryptography background

The most important task of cryptography is the encryption of data. Encryption is the process of transforming data into other form, unreadable to anyone except of those possessing special knowledge. In this case information is called *plain text* and its unreadable form is called *ciphertext*. If we want to encrypt plaintext (or decrypt ciphertext) we need the key, which must be used with an appropriate algorithm—called *cipher* [1].

4.1 Basic functions

Contemporary ciphers (particularly symmetric encryption/decryption algorithms) are based on few simple functions. In this section, we present substitution and permutation as well as their combination.

- **Simple substitution**

In this method, each character of the plaintext is substituted by any other character. The only condition is that two different characters must not be substituted by the same letter.

Let us assume that a *substitution* is presented as: $\pi: \Sigma \rightarrow \Sigma$

Then, a plaintext: $a_1a_2\dots a_n$ ($a_i \in \Sigma$) is encrypted as: $\pi(a_1)\pi(a_2)\dots\pi(a_n)$

And a ciphertext: $b_1b_2\dots b_n$ is decrypted as: $\pi^{-1}(b_1)\pi^{-1}(b_2)\dots\pi^{-1}(b_n)$

As a simple example, we can assume that the main text is the word SUBSTITUTE, and a possible substitution scheme is presented in Table 4.1.

Table 4.1: An example of substitution table

S	A
U	K
B	Q
T	R
I	G
E	S

Then, the resulted text after the replacement is AKQARGRKRS. Unfortunately, the substitution is very vulnerable to statistical cryptanalysis, and it may be used as a helpful component but only in combination with more complex methods.

- **Substitution Box (S-BOX)**

Substitution matrix or Substitution Box (S-BOX) is a table consisting of n rows and m columns. This table converts input bits into output bits. Let us consider the S-box which operates on 8-bit data blocks: first 4 bits from the block define the row (r), and the following 4 bits define the column (c). The block is substituted by element taken from S-box which coordinates are (r, c) . That element also has an 8-bit length. This matrix is *bijective*, which means that for each two different blocks of data we get different results of the substitution. An example of such matrix is presented in Table 4.2.

Table 4.2: An example of S-box

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	c6	07	ee	f6	1a	29	de	8b	60	fd	31	56	02	af	a8	13
1	95	05	93	fa	f5	b2	8e	e1	a4	a9	ba	5f	39	b6	e4	81
2	6f	04	27	b3	6c	7e	10	99	97	4b	34	1c	e2	b1	9d	d5
3	f7	70	b9	87	30	2d	0a	35	f1	24	fe	c5	d7	4e	65	15
4	12	f8	a7	cb	36	23	b4	41	5b	89	52	98	ad	38	a1	09
5	a6	a3	00	db	bf	f9	63	49	d4	68	7d	72	6b	67	4f	9f
6	5e	20	55	08	79	9a	66	f4	75	f3	fb	01	a0	78	3f	ae
7	5d	47	7f	e0	da	c4	8f	eb	86	92	4a	42	df	ff	e7	a5
8	64	18	d9	26	be	d0	88	2e	76	b0	fc	85	37	45	cd	19
9	c0	03	61	46	44	ab	21	11	73	dd	71	28	bd	43	e9	b7
A	3e	9b	74	14	6d	0c	48	b8	7a	58	59	3b	dc	2b	c9	0d
B	cf	6e	91	25	1b	54	9c	53	d8	ac	16	2a	ca	0b	a2	ef
C	8a	f0	b5	5c	c7	4d	69	8d	d1	bb	e8	c1	96	7b	17	ea
D	1f	83	94	cc	90	06	ed	e3	3d	6a	51	8c	f2	7c	3a	c2
E	c3	0e	ce	22	d2	4c	1d	d6	c8	3c	0f	2c	62	aa	50	40
F	e6	bc	ec	e5	80	32	84	2f	82	33	5a	1e	9e	57	77	d3

Now, let us consider an example of substitution by means of the S-box presented in Table 4.2:

- **Input data block:** 1001 1101_(BIN)
 Row number: 1001_(BIN) (9_(HEX))
 Column number: 1101_(BIN) (D_(HEX))
- **Substituted element:** 43_(HEX) (0100 0011_(BIN))

Thus, the obtained substitution is: 1001 1101_(BIN) → 0100 0011_(BIN)

It is important that the substitution matrix guarantees a high resistance to cryptanalysis. Good S-boxes ensure non-linearity, and a flat XOR distribution provides resistance to differential cryptanalysis.

- **Permutation**

Permutation is a mapping of biunique finite set of itself, or simply put to mix characters in a given set of characters. Permutation is widely used in symmetric algorithms, but it is extremely vulnerable to differential cryptanalysis if used by itself. In general, a *permutation function* is a function that permutes (reorganizes, mixes) all bits of the data—we present a simple example in Figure 4.1.



Figure 4.1: An example of a complex permutation

This function operates on 16 bit data blocks. In practice, only functions that operate on greater blocks are used. In these algorithms the actual minimum block length are assumed to be 256 bits. The permutation function presented in Figure 4.1 gives the impression of having no pattern in reorganization of successive bits, what causes the fact that the attack on the cipher have to be more complex and difficult to carry out. Let's name this function as a complex permutation. We can also define a simple permutation that possess a less complicated reorganization scheme, i.e. like the simple rotation of two bits to the right presented in Figure 4.2.

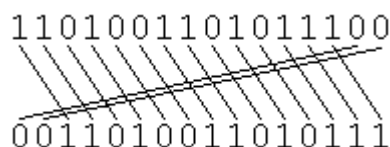


Figure 4.2: An example of a simple permutation

This mechanism is fast in realization, but it does not increase the complexity of whole encryption round. He only thing that have in common these two permutation functions is the fact that they are assigning a new position for every bit of the input data block.

Let us look at the substitution matrix from a different point of view. It is used for the substitution, but it can be also used for permutation needs. S-Box possesses 256 different input values and 256 different output values and it is also bijective. Let us assume that the input value will indicate the bit position in a 256 block sequence and the output value will indicate its position in the reorganized word. Let us consider an example when we use the S-box given in Table 4.2:

Bit 0 (input value 00_(HEX)) will be placed in the 198th position (output value C6_(HEX))

Bit 1 (input value 01_(HEX)) will be placed in the 7th position (output value 07_(HEX))

...

Bit 255 (input value FF_(HEX)) will be placed in the 211th position (output value D3_(HEX))

This concludes the use of substitution box as a base for permutation function.

- **Exclusive OR**

Exclusive OR (XOR) is a basic operation between two binary numbers. The principle of this operation is shown in Table 4.3.

Table 4.3: Exclusive OR

a	b	$\oplus$
0	0	0
0	1	1
1	0	1
1	1	0

- **Substitution-permutation network**

A substitution-permutation network is another solution that is used by block ciphers. This network operates on substitution (usually S-boxes) and permutation functions. At each round, the round key is combined using some group operation, typically XOR.

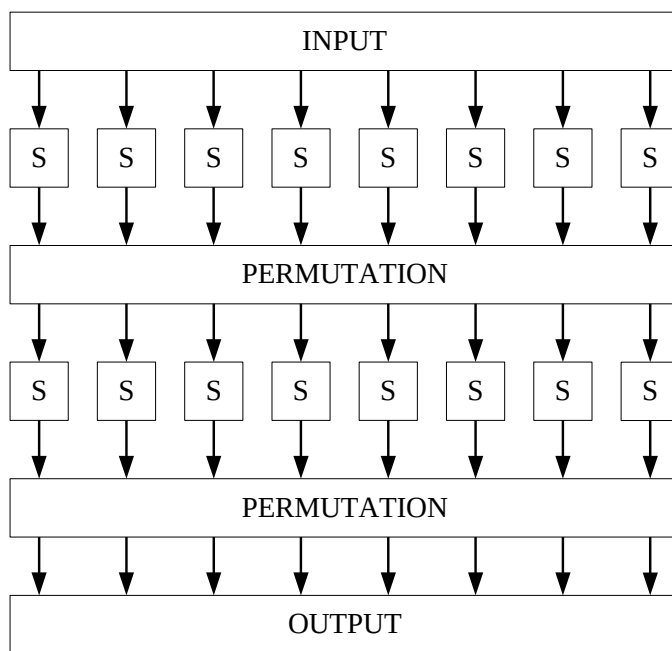


Figure 4.3: An example of substitution-permutation network

Figure 4.3 presents an example of substitution-permutation network—a sequence of substitution and permutation operations.

4.2 Security features

In this section we present some security features that are crucial to cryptography. Presented features determine the security level of the considered encryption/decryption algorithm.

- **Nonlinearity**

The function that meets this criterion is one that is not linear. Linear functions satisfy the following properties:

$$\begin{aligned} \text{Additivity: } f(x) + f(y) &= f(x + y) \\ \text{Homogeneity: } \alpha f(x) &= f(\alpha x) \end{aligned}$$

To describe the degree of nonlinearity, we must define a Boolean function f with n variables, which is a transformation from $\mathbb{F}_2^n$ to $\mathbb{F}_2$. A Boolean function is linear when, for any $\alpha \in \mathbb{F}_2$, φ_α , it is a linear function of n variables:

$$x \mapsto \alpha x$$

For any Boolean function, the following value is related to the Walsh transform:

$$\sum_{x \in \mathbb{F}_2^n} (-1)^{f(x)} = 2^n - 2wt(f)$$

Where $wt(f)$ is the Hamming weight of f .

The Walsh spectrum of a Boolean function:

$$\{\mathcal{F}(f + \varphi_\alpha), \alpha \in \mathbb{F}_2^n\}$$

$$x \mapsto \alpha x$$

The Walsh spectrum of a vectorial function F from $\mathbb{F}_2^n$ to $\mathbb{F}_2$:

$$\{\mathcal{F}(\varphi_\alpha \circ F + \varphi_\beta), \alpha \in \mathbb{F}_2^n \setminus \{0\}, \beta \in \mathbb{F}_2^n\}$$

Where:

$$\varphi_\alpha \circ F: x \mapsto \alpha \cdot F(x)$$

For linear functions $x \mapsto \varphi_\alpha \circ F(x) + \varphi_\beta(x)$ take the same values for x , but for nonlinear functions this relationship does not hold. In this case $\varphi_\alpha \circ F_K$ should be far from affine functions.

Nonlinearity of a function is a function F from $\mathbb{F}_2^n$ to $\mathbb{F}_2$, which is the Hamming distance between all $\varphi_\alpha \circ F, \alpha \in \mathbb{F}_2^n, \alpha \neq 0$ and the set of affine functions.

$$2^{n-1} - \frac{1}{2} \mathcal{L}(F) \text{ where } \mathcal{L}(F) = \max_{\alpha \in \mathbb{F}_2^n} \max_{\beta \in \mathbb{F}_2^n} |\mathcal{F}(\varphi_\alpha \circ F + \varphi_\beta)|$$

For any function $F: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ that satisfies: $\mathcal{L}(F) \geq 2^{\frac{n+1}{2}}$ in case of equality function is *almost bent*. Bent means that function is perfectly nonlinear.

- **The Strict Avalanche Criterion (SAC)**

It is satisfied if for all $i, j \in \{0, 1, \dots, t-1\}$, input bit i changes output bit j with probability exactly 0,5.

- **Completeness**

Bijjective function $f: \{0, 1\}^t \mapsto \{0, 1\}^t$ is complete if for all $i, j \in \{0, 1, \dots, t-1\}$ exists a vector $S(X)$ such that $S(X \oplus e_j)$ and $X \in \{0, 1\}^t$ differ in at last bit j . e_j is t -bit unit vector with “1” in position i . In other words, a function is complete, when every output bit depends upon every input bit.

- **Diffusion Order**

It ensures that even if the value of the output bits that change is large, the number of changes in the entry is relatively low. Diffusion order specifies the minimum number of changes to the entry, which occurs when a single input bit changes. Formally:

$$wt(dY) > \lambda + 1 - wt(dX) \text{ for } wt(dX) < \lambda + 1,$$

where λ is the diffusion order.

- **Low XOR Table**

For any $n \times n$ S-box, S-box XOR table entries are defined through position (i, j) in the XOR table, which contains value:

$$|\{X \in \{0, 1\}^n: S(X) \oplus S(X \oplus i) = j\}|$$

This model is determined by the number of pairs of input values (row), which gives the output value (column). S-box is secure when it has low XOR table entries, ideally — “0” or “2”.

4.3 Operation modes

Block ciphers operate on blocks of strictly defined length. If we want to encrypt plaintext that is longer than a single block, we must use an appropriate operation mode. These modes specify how a sequence of plaintext should be encrypted. Below, we present the different modes on how to give to the input and output a block of data. Although, block ciphers can use many different operation modes (cipher feedback, output feedback, and many others), in this section we describe the modes that we had implemented and tested in the application presented in the deliverable D9.13: electronic codebook and cipher block chaining.

• ECB (Electronic Codebook)

Each of the blocks is encrypted independently of the others, using the same key. Damage (error) of a single block does not affect other blocks— this is one of the biggest advantages of the ECB mode. Besides, this method is faster than the others, because of its simplicity. It also allows users to decrypt a specific block of data without decrypting any other.

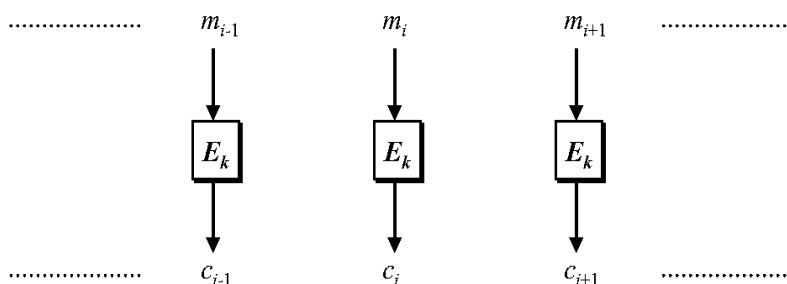


Figure 4.4: Ciphering in ECB mode

• CBC (Cipher Block Chaining)

This algorithm adds (XOR) input blocks with a previously encrypted block. Such correlation causes that an error in one block propagates errors in another block. An additional advantage is that two identical plaintexts can be transformed into different ciphertexts if they have different initialization vectors (c_0).

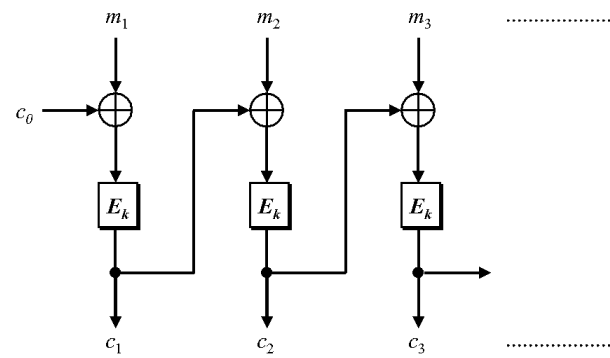


Figure 4.5: Ciphering in CBC mode

5 Contemporary block ciphers

Nowadays, cryptography is the basic technique that ensures data confidentiality. Symmetric block ciphers, thanks to their high level of security, good performance and ease of construction, are very popular solutions in network environments. We know a lot of different encryption/decryption algorithms, but only a few ciphers are usually used [2]. In this chapter, the most popular symmetric block ciphers are presented.

5.1 DES

The Data Encryption Standard (DES) was approved in 1976. This algorithm was the basis for the creation of modern methods. The strength of this method is the irreversible S-box. DES operates on 64-bit blocks and uses 56-bit key (created from 64 bits). Operations are performed in key-dependent rounds using Feistel Networks. Below its main characteristics describe how DES encryption works.

1. Input Permutation – this process serves for no security purpose. It is only due to hardware issues. In hardware we can easily implement permutation, but in software it is harder because of efficiency.
2. Key generation – before each round a key is obtained from the 56-bit main key. This key is decomposed into two 28-bit halves and each half is rotated by 1 or 2 bits (depending on round).
3. Function f (16 rounds):
 - a. Right half of block R_i is expanded from 32 to 48 bits
 - b. Key and R_i are XORed
 - c. 48-bit sequence is transformed by S-box
 - d. 32-bit is permuted by P-box
4. Output permutation.

According to Feistel Network theory, the decryption operation is similar to encryption. The algorithm must only reverse key and permutations.

DES properties:

- Ease of implementation in hardware (only shifts, permutations and XOR operations).
- The S-boxes cause non-linearity and resistance to differential cryptanalysis.
- The expansion permutation and P-boxes are responsible for avalanche effect.
- Each bit of key has an important influence to ciphertext (many rounds with different key combination).

5.2 Triple DES

One of DES disadvantages is its short key, thus triple-DES (3DES) method was proposed. In this algorithm we need two or three different keys. Let us consider the triple-DES with two different keys. At the beginning we use K to encrypt plaintext, and then use the second key K' to decrypt ciphertext. Finally we encrypt sequence of bit another time by key K. Formally we can present this idea as:

$$C = DES_K(DES_{K'}^{-1}(DES_K(P)))$$

Triple-DES is much secure algorithm, but is rather slow (especially in software). For this reason DES (3DES) has been replaced by more modern and faster algorithm (mainly AES).

5.3 IDEA

IDEA—International Data Encryption Algorithm—was created in 1991 by Xuejia Lai and James Massey. The cipher is patented, but is free for no-commercial use. IDEA was to be successor to DES, but problems with licenses caused the loss of popularity.

IDEA operates on 64-bit blocks and uses a 128-bit key, which is used to generate 52 subkeys (16-bit). Main key is divided into 8 keys, and then it is rotated by 25 bits and once again decomposed. This method is repeated until 52 subkeys are generated. Points below describe how IDEA works.

1. Blocks are split into four subblocks of 16-bits, and then the eight-round algorithm is applied to them.
2. In each one of 20 rounds, blocks are linked with three operations:
 - a. XOR
 - b. Modulo 2^{16} addition
 - c. Modulo $2^{16}+1$ multiplication
3. It uses six keys in every round.
4. The four subblocks are linked with the four remaining subkeys and composed into one single 64-bit ciphertext block.

Decryption of the IDEA requires changes in the Algorithm. We need to change the order of subkeys (as in DES algorithm) and also perform such modifications as:

- Reverse keys used for the multiplication
- Negate keys used for the addition

5.4 AES

AES—Advanced Encryption Standard—was originally published as Rijndael in 1998 but it was later announced by the National Institute of Standards and Technology (NIST) as a standard in 2001 [3]. Rijndael was developed by two Belgian

cryptographers J. Daemen and V. Rijmen, this cipher was the winner of the contest, which was to determine a DES successor.

AES is based on Substitution-permutation network and operates on 128, 192 and 256-bit blocks. An improvement over other ciphers is that it is possible to use different key lengths. We can use the keys with a length equal to: 128, 192 or 256-bit, which determines the number of rounds. Each block is placed into a 4x4 matrix in which operations are performed. In each round four operations on the 4x4 matrix are applied:

- `ByteSub()` – substitution function performed by the special S-box.
- `ShiftRows()` – bytes in the last three rows are shifted over different numbers of bytes (Figure 5.1).

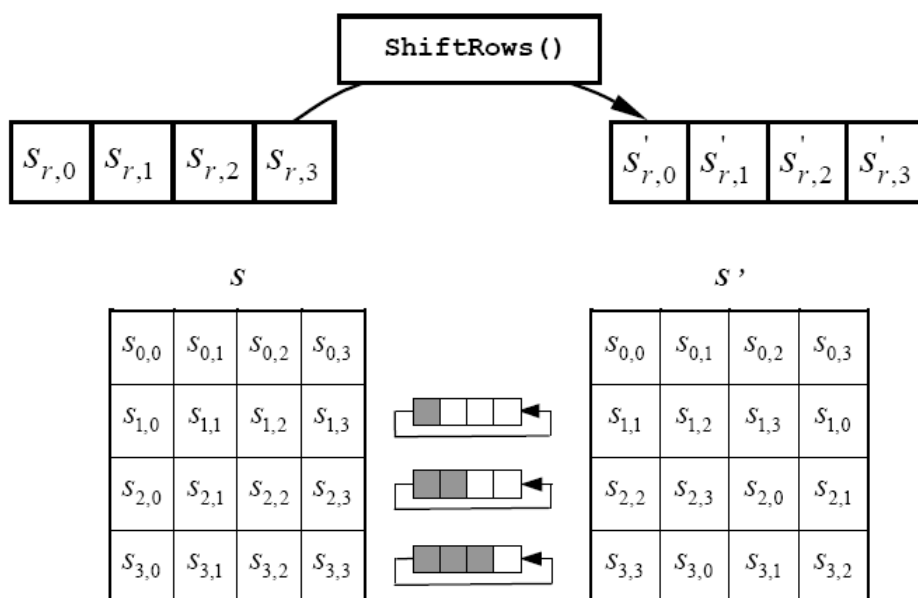


Figure 5.1: ShiftRows() operation (The Figure comes from: [3])

- `MixColumn()` – columns are considered as polynomials over $GF(2^8)$ and multiplied modulo x^4+1 with a fixed polynomial $a(x)$ (Figure 5.2). The acronym GF means Galois field (a field that contains only finitely many elements).

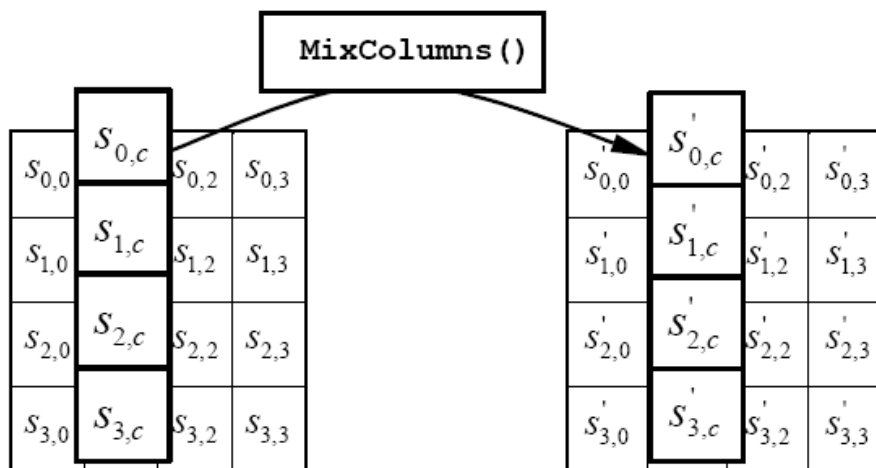


Figure 5.2: MixColumn() operation (The Figure comes from: [3])

- $\text{AddRoundKey}()$ – round key is added (XOR) to the matrix.

Key generation is using special algorithm, which is called Key Expansion. This method creates one key for each round.

Let us consider an example of AES cipher: AES-128. In this case algorithm consists of 10 rounds and one round zero. In round zero only the $\text{RoundKey}()$ operation is performed. The remaining ten iterations contain all four operations, which are described earlier. This example is presented in the Figure 5.3.

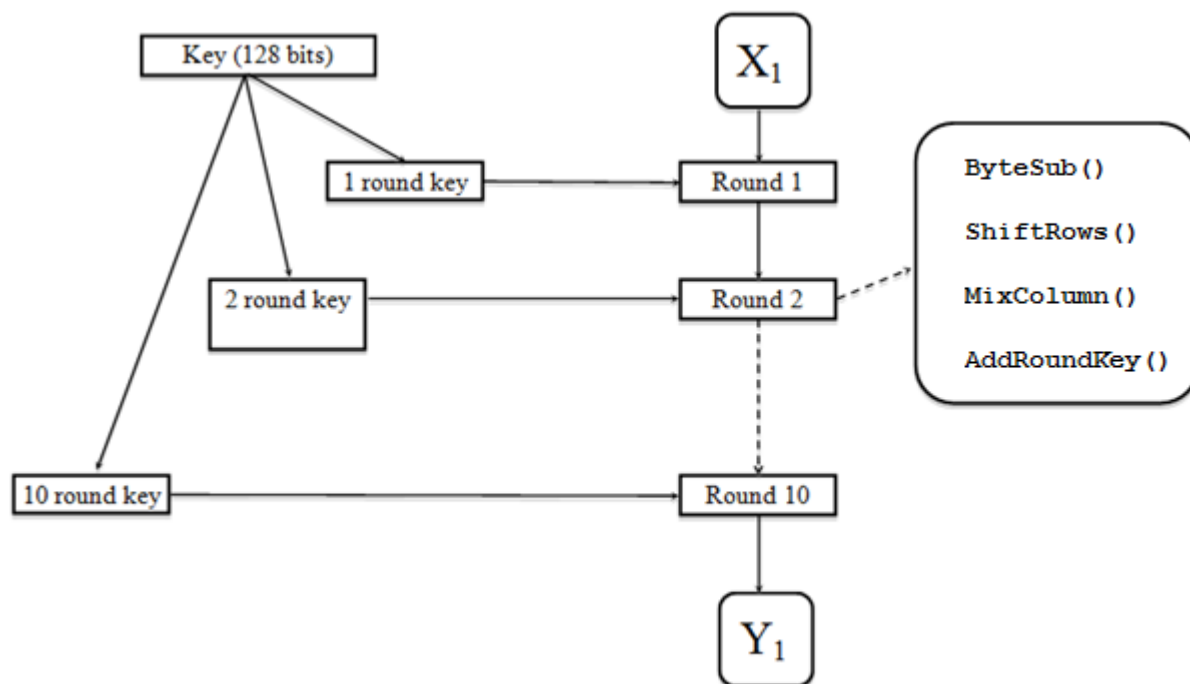


Figure 5.3: Basic representation of AES algorithm

Decryption of the AES requires definition of the equivalent inverse cipher and reverse order of operations. Decryption process consists of operations such as:

- `InvByteSub()`
- `InvShiftRow()`
- `InvMixColumn()`
- `AddRoundKey()`

6 New block cipher

In this chapter the new symmetric block cipher is presented in detail. The general structure of the cipher is presented, as well as the crucial ideas of its design (i.e. 'basic function' in S-box). In the end of this chapter we present the whole encryption process.

6.1 General structure

The proposed block cipher is based on two functions: substitution matrix (S-Box) and permutation. These operations are used in each round of the cipher. The overall idea of a single round is presented in Figure 6.1.

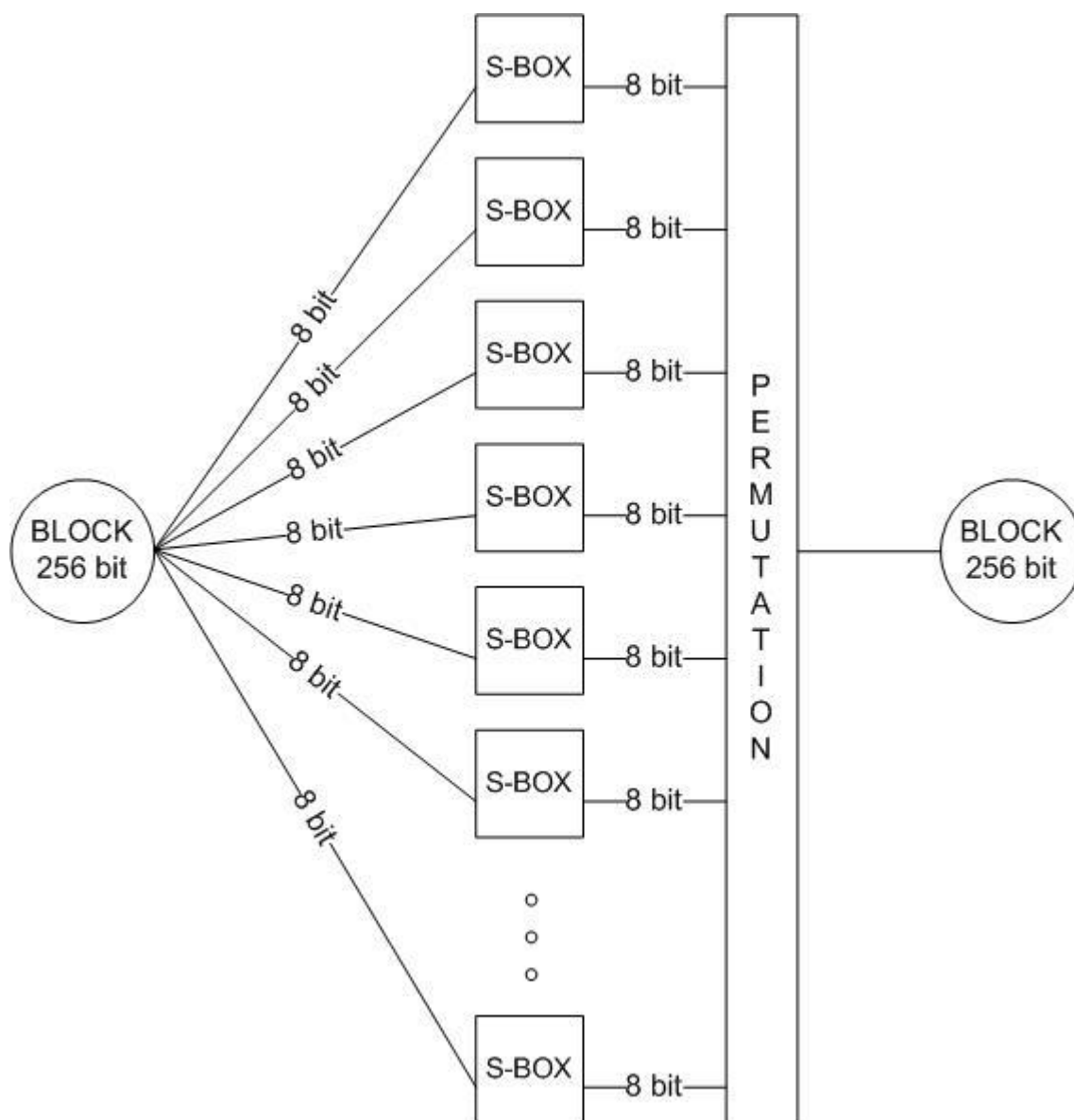


Figure 6.1: The diagram of a single round of the new cipher

The 256-bit block is divided into 64 sub-blocks 8-bit each (for clarity only 8 sub-blocks are presented in the diagram). In the next step each sub-block is applied to the substitution box as an input value (details about this function are presented in subsection 4.1). Output values are concatenated into one 256-bit block (method of merging is adequate to the division schema). The last step is the use of permutation function (based on S-Box) on the 256-bit block of data. In this point, algorithm repeats. An example of a single round in new cipher is presented in Figure 6.2.

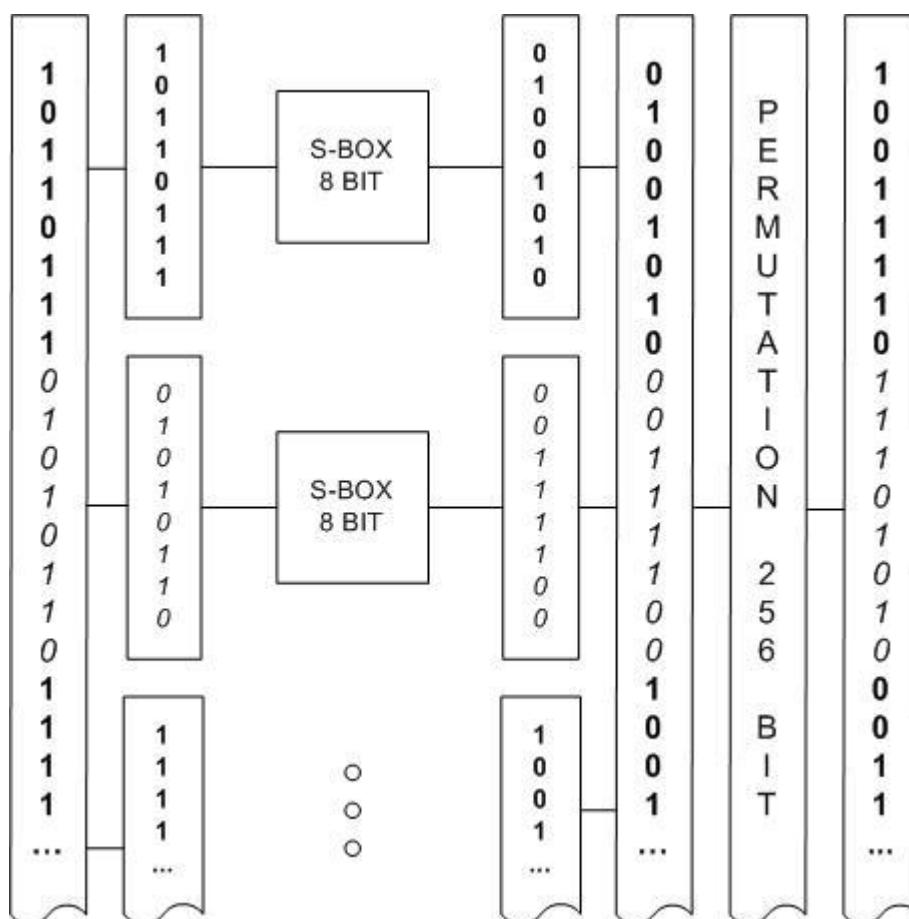


Figure 6.2: Example of a single round of the new cipher

Below, we present the general features of new cipher:

- Block size: 256 bit
- Key lengths: 128, 192, 320, 576 bit
- Number of rounds (corresponding to key lengths in order): 8, 10, 12, 14

6.2 The idea of ‘basic functions’

In cryptographic algorithms the key is usually a block of bits, which is often XORed with partially encrypted data in a given phase of algorithm’s round, and the rest of round’s phases are fully independent from the key. However, there is a possibility of making these phases to also rely on the key. Substitution phase, as well as permutation phase is based on S-Boxes. We can use S-box taken from AES algorithm (because of excellent security parameters) for the need of creating a relation between the key and the generation of substitution matrices. The S-box used by the AES cipher is presented in Table 6.1.

Table 6.1: S-Box used in AES

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
A	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
B	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
C	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
D	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
E	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
F	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Each element of this matrix can be interpreted as a 8-bit sequence in binary system. We can now isolate 8 functions: $GF(2^8) \rightarrow GF(2)$. From each element of the matrix (output value) we take the proper bit and assign it to the input value (which remains the same). Because the sequences are 8-bit long there are 8 different functions that we are able to isolate. We can write the S-box used in the AES cipher (presented in the Table 6.1) in binary form—presented in the Table 6.2. Then we can write all first bits with one color, all second bits with another color etc. Let us call this functions as basic functions and denote them as F0 (this basic function consists of red colored bits presented in the Table 6.2), F1 (green color in the Table 6.2), F2 (blue color in the Table 6.2), F3, F4, F5, F6, F7.

Table 6.2: S-Box used in AES

	0	1	2	3		E	F
0	01100011	01111100	01110111	01111101		10101011	01110110
1	11001010	10000010	11001001	01111101		01110010	11000000
2	10110111	11111101	10010011	00100110		00110001	00010101
3	00000100	11000111	00100011	11000011		10110010	01110101
4	00001001	10000011	00101100	00011010		00101111	10000100
5	01010011	11010001	00000000	11101101		01011000	11001111
6	11010000	11101111	10101010	11111101		10011111	10101000
7	01010001	10100011	01000000	10001111		11110011	11010010
8	11001101	00001100	00010011	11101100		00011001	01110011
9	01100000	10000001	01001111	11011100		00001011	11011011
A	11100000	00110010	00111010	00001010		11100100	01111001
B	11100111	11001000	00110111	01101101		10101110	00001000
C	10111010	01111000	00100101	00101110		10001011	10001010
D	01110000	00111110	10110101	01100110		00011101	10011110
E	11100001	11111000	10011000	00010001		00101000	11011111
F	10001100	10100001	10001001	00001101		10111011	00010110

Let us consider an example:

First basic function (F0) created from bits at position 0:

$S\text{-Box}(00_{(\text{HEX})}) = 63_{(\text{HEX})} = 01100011_{(\text{BIN})}$ therefore $F0(00_{(\text{HEX})}) = 1$
 $S\text{-Box}(01_{(\text{HEX})}) = 7c_{(\text{HEX})} = 01111100_{(\text{BIN})}$ therefore $F0(01_{(\text{HEX})}) = 0$
 $S\text{-Box}(02_{(\text{HEX})}) = 77_{(\text{HEX})} = 01110111_{(\text{BIN})}$ therefore $F0(02_{(\text{HEX})}) = 1$
 $S\text{-Box}(03_{(\text{HEX})}) = 7b_{(\text{HEX})} = 01111011_{(\text{BIN})}$ therefore $F0(03_{(\text{HEX})}) = 1$

$S\text{-Box}(FF_{(\text{HEX})}) = 16_{(\text{HEX})} = 00010110_{(\text{BIN})}$ therefore $F0(FF_{(\text{HEX})}) = 0$

Sixth basic function (F6) created from bits at position 6:

$S\text{-Box}(00_{(\text{HEX})}) = 63_{(\text{HEX})} = 01100011_{(\text{BIN})}$ therefore $F6(00_{(\text{HEX})}) = 1$
 $S\text{-Box}(01_{(\text{HEX})}) = 7c_{(\text{HEX})} = 01111100_{(\text{BIN})}$ therefore $F6(01_{(\text{HEX})}) = 1$
 $S\text{-Box}(02_{(\text{HEX})}) = 77_{(\text{HEX})} = 01110111_{(\text{BIN})}$ therefore $F6(02_{(\text{HEX})}) = 1$
 $S\text{-Box}(03_{(\text{HEX})}) = 7b_{(\text{HEX})} = 01111011_{(\text{BIN})}$ therefore $F6(03_{(\text{HEX})}) = 1$

$S\text{-Box}(FF_{(\text{HEX})}) = 16_{(\text{HEX})} = 00010110_{(\text{BIN})}$ therefore $F6(FF_{(\text{HEX})}) = 0$

Having 8 basic functions allows us to create new functions by XOR-ing the output values of the different basic functions. Newly created functions are nothing more than linear combinations of basic functions (let us denote them as LC). Numbers of linear combinations correspond to the basic functions from which they were created. For example: LC34 is created by operation of XOR-ing F1 and F5 (i.e. $34 = 00100010_{(\text{BIN})}$).

Let us consider an example. When we want to create LC56, we represent the decimal number 56 in binary format:

$$56 = 00111000_{(\text{BIN})}$$

We take into account the bits set to '1' and then we obtain linear combination LC56 as created by XOR-ing F3, F4 and F5 (the functions where we have bits set to '1'). Now let us try to find result of this LC56 function for the input value of $EB_{(HEX)}$. Output value of AES S-Box for the $EB_{(HEX)}$ value as an input is $E9_{(HEX)} = 11101001$.

$$\begin{aligned} F3(EB_{(HEX)}) &= 1 \\ F4(EB_{(HEX)}) &= 0 \\ F5(EB_{(HEX)}) &= 1 \end{aligned}$$

Therefore, after XORing values above we get $LC56(EB_{(HEX)})=0$.

By having 8 basic functions we can create 2^8-1 linear combinations (LC0 is $GF(2^8) \rightarrow 0$ and it is therefore useless). It should be noted that the collection of linear combinations also contains the basic functions $LC1()=F0()$, $LC2()=F1()$, $LC4()=F2()$, $LC8()=F3()$, $LC16()=F4()$, $LC32()=F5()$, $LC64()=F6()$ and $LC128()=F7()$. Non-linearity of every one of the LC functions is equal to 112, which has been tested by means of the simulator described in Chapter 7.

6.3 New substitution boxes

In order to create a substitution box, it is required to use 8 LCs taken from the set of 255 possible ones. However these LCs must not be freely chosen, because it could result in a situation in which the matrix would lose its bijectivity. As it turns out, the linear combinations chosen to create a S-box have to be mutually non-linear. It means that none of the used LCs could be created by XORing any number of other 7 LCs.

Proof:

For simplification, the proof is conducted on substitution matrix of size 4x4, whose elements belong to $GF(2^4)$. An example of such matrix (in hexadecimal format):

$$\begin{bmatrix} A & 0 & D & 3 \\ 5 & 4 & 6 & C \\ 1 & F & E & 8 \\ 2 & 7 & 9 & B \end{bmatrix} \quad (3)$$

Each value of the matrix can be represented by a third degree polynomial belonging to the $GF(2^4)$:

$$ax^3 + bx^2 + cx + d; a, b, c, d \in GF(2) \leftrightarrow \{0,1\} \quad (4)$$

$$e.g.: 9_{(HEX)} = 1001_{(BIN)} \rightarrow x^3 + 1 \quad (5)$$

For each of these polynomials we can find other polynomials in the form of:

$$\bar{a}x^3 + bx^2 + cx + d \quad (6)$$

$$ax^3 + \bar{b}x^2 + cx + d \quad (7)$$

$$ax^3 + bx^2 + \bar{c}x + d \quad (8)$$

$$ax^3 + bx^2 + cx + \bar{d} \quad (9)$$

Let us consider the pair of equations (4) and (6). In the 4x4 matrix we are able to find 8 pairs of the type: $0000_{(BIN)}, 1000_{(BIN)}$; $0001_{(BIN)}, 1001_{(BIN)}$; $0010_{(BIN)}, 1010_{(BIN)}$; $0011_{(BIN)}, 1011_{(BIN)}$; etc. ($0_{(HEX)}, 8_{(HEX)}$; $1_{(HEX)}, 9_{(HEX)}$; $2_{(HEX)}, A_{(HEX)}$; $3_{(HEX)}, B_{(HEX)}$; ...) which differ in the first bit. For these pairs we create a polynomial which a coefficient translates into a XOR e, where e is equal to b, c, d or a linear combination of these, but it is constant for every of these pairs (in case when for one pair selected $e=c$, for every pair e equals c). In this case, it is important that e is linearly independent from a (in other case polynomials in given pair will become identical). Pair (4), (6) is then substituted by pair:

$$(a \oplus e)x^3 + bx^2 + cx + d \quad (10)$$

$$(\bar{a} \oplus e)x^3 + bx^2 + cx + d \quad (11)$$

And also:

$$(a \oplus e)x^3 + bx^2 + cx + d \wedge e = 0 \Leftrightarrow ax^3 + bx^2 + cx + d \quad (12)$$

$$(\bar{a} \oplus e)x^3 + bx^2 + cx + d \wedge e = 0 \Leftrightarrow \bar{a}x^3 + bx^2 + cx + d \quad (13)$$

$$(a \oplus e)x^3 + bx^2 + cx + d \wedge e = 1 \Leftrightarrow \bar{a}x^3 + bx^2 + cx + d \quad (14)$$

$$(\bar{a} \oplus e)x^3 + bx^2 + cx + d \wedge e = 1 \Leftrightarrow ax^3 + bx^2 + cx + d \quad (15)$$

As we can see, the polynomial will remain unchanged ([10],[11]) or it will become the second polynomial from the pair ([12],[13]) so the bijection will be preserved. In this point we obtain a new 4x4 matrix. For a 4x4 matrix there are 15 linear combinations, e.g.: linear combination #13 ($1101_{(BIN)}$) is no different from $a \oplus b \oplus d$. For pairs [4][7], [4][8], [4][9] the proof is analogous, and can be also generalized and applied to the $2N \times 2N$ matrices (elements of matrices belonging to $GF(2^{2N})$).

Therefore linear combinations are required to be chosen in a way that guarantees their linear independence. Method of choosing LCs:

- 1) First LC can be chosen from the full set – It gives us 255 possible choices. There are no restrictions here.
- 2) Second LC can be chosen from the remaining 254, because we cannot take the previously chosen LC.
- 3) Third LC can be chosen on 252 different ways. LCs that we cannot choose are neither the two previously chosen nor a third one, which is their XOR.
- 4) Fourth on 248 ways. Eliminated: three already chosen, XORs of any two (there are 3) and XOR of all of the three already chosen, which gives 7 eliminated LCs.
- 5) Fifth on 240 ways. Eliminated: four already chosen, XORs of any two (there is 6), XORs of any three (there are 4) and XOR of all of the four already chosen, which gives 15 eliminated LCs.
- 6) Sixth on 224 ways. Eliminated: five already chosen, XORs of any two (there is 10), XORs of any three (there are 10), XORs of any four (there are 5) and XOR of all of the five already chosen, which gives 31 eliminated LCs.
- 7) Seventh on 192 ways. Eliminated: six already chosen, XORs of any two (there are 15), XORs of any three (there are 20), XORs of any four (there are 15),

XORs of any five (there are 6) and XOR of all of the six already chosen, which gives 63 eliminated LCs.

- 8) Eighth on 128 ways. Eliminated: seven already chosen, XORs of any two (there are 21), XORs of any three (there are 35), XORs of any four (there are 35), XORs of any five (there are 21), XOR of any six (there are 7) and XOR of all of the seven already chosen, which gives 127 eliminated LCs.

Maximum number of S-boxes that we can obtain this way is equal to the multiplication of number of ways that every bit can be chosen. This equals to:

$$255 \cdot 254 \cdot 252 \cdot 248 \cdot 240 \cdot 224 \cdot 192 \cdot 128 = 5348063769211699200 \text{ (about } 5,35 \cdot 10^{18} \text{)}$$

As we can see, this gives us a huge number of substitution boxes. However, all of them share the same common mutual flaw, because the original AES S-Box. Every one of these S-Boxes for the input value of 52_(HEX) results in an output value of 00_(HEX). This is due to the fact that for every linear combination the following is always true: $LC_n(52)=00$. Each one of these S-boxes can be coded with 63 bits, but the use of 64 bits is optimal (a 8-bit number of every linear combination).

But we can even use the 64th bit better—in order to increase the number of S-boxes twice (about 10^{19} S-boxes). In addition to the standard AES S-box, we can also use the inverted AES S-box (presented in Table 6.3) as the base for creating new S-boxes. We assumed that if last bit equals 0 we use standard AES S-box and when last bit equals 1, then invert AES S-box is used. S-boxes created on this way (base: inverted AES S-box) have their own flaw: input 63_(HEX) always results value 00 in output.

Table 6.3: Inverted S-Box used in AES

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
4	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
9	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
A	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
B	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
C	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
D	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
E	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
F	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

6.4 Keys

Coding every S-Box created by isolating linear combinations takes 64 bits. Therefore, the key has to be a multiplication of 64 bits. The smallest possible value to obtain a key is 64 bits, assuming that every 8 bit block in the second phase of algorithm is coded with the same substitution box that will be used as a permutation function's base. The largest key is $65 \times 64 = 4160$ bits, when each one of 8-bit 64 blocks in the second phase is substituted with different S-box and the permutation function's base is also an unique S-box. Keys whose length lies in between the smallest and the largest key are also possible to use (but they still are required to be a multiple of 64 bits).

6.5 The algorithm

In this section we present the encryption/decryption algorithm.

1. First we choose the key length. We have 4 key lengths chosen for practical use. Each of those keys codes certain number of matrices (S-boxes). Each of them are code-able by a 64 bit chain (key values must be a multiple of 64). Available key lengths:
 - **128 bits** (2 S-boxes are coded: one for substitution transformation and another for permutation transformation. With 128-bit key we propose 8 rounds of the algorithm)
 - **192 bits** (3 S-boxes are coded: two for substitution transformation and one for permutation transformation. We propose that the number of rounds equals 10)
 - **320 bits** (5 S-boxes are coded: four for substitution transformation and one for permutation transformation. We propose that the number of rounds equals 12)
 - **576 bits** (9 S-boxes are coded: eight for substitution transformation and one for permutation transformation. We propose that the number of rounds equals 14)
2. The S-boxes are generated. The number of generated matrices corresponds to the key length. First 63 bits of the 64 bit key part describes what linear combinations are used, while the last bit marks which S-box is used: standard AES S-box or inverse S-Box.
3. Plaintext is separated in 256 bit blocks.
4. Whole 256 bit block is permuted by the last S-box (the same step as presented in the point 7). This additional permutation eliminates the flaw considered in the section 7.1 (mapping $52_{(HEX)}$ to $00_{(HEX)}$ or $63_{(HEX)}$ to $00_{(HEX)}$).

5. Algorithm splits each 256 bit block into 8 bit sub-blocks for substitution (shown in Figure 6.2). Depending on key lengths:
 - For a 128 bit key, all 8 bit sub-blocks are substituted by one S-box,
 - For a 192 bit key, first 64 8 bit sub-blocks are substituted by one S-box, and second 64 8 bit sub-blocks are substituted by 2nd S-box,
 - For a 320 bit key, we have similar situation but we now have 4 substitution S-boxes and four 32x8 bit sub-block parts for each S-box,
 - For a 576 bit key, we have eight 16x8 bit sub-block parts for substitution.
6. Now all 8 bit sub-blocks are merged into 256 bit block (shown in Figure 6.2).
7. Whole 256 bit block is permuted by the last S-box:
The number of the bit in 256 bit block is substituted in the last S-box, and the outcome number marks the destination spot.
8. The next round begins (point 5).

If the length of the last block of plain text is smaller than 256 bits it is padded with zeroes and then encrypted like any other block.

7 Evaluation of security and vulnerabilities

This chapter is dedicated to the evaluation of the proposed block cipher. It also contains the vulnerability considerations of new solution. We consider the cryptanalysis as well as the simulator, developed in WP8, which is able to test and evaluate single S-boxes and ciphertexts.

7.1 Cryptanalysis

Cryptanalysis is the study of methods which allow finding a plaintext or secret key. Finding a way (method) that allows finding the secret key in a quicker way than searching the whole key space (brute force) is in fact breaking the cipher. Almost every method of cryptanalysis can be qualified as one of the following attack types:

- *Ciphertext-only attack* – in this type of attack we have access only to set of ciphertexts,.
- *Known-plaintext attack* – here we have access to both plaintext and its ciphertext.
- *Chosen-plaintext attack* – attacker have capability to choose plaintexts and cipher them with searched key.
- *Chosen-ciphertext attack* – attacker can choose ciphertexts and decipher them using the searched key.
- *Adaptive chosen-ciphertext attack* – like the above one but, thanks to knowledge about results of the decryption, we can choose subsequent ciphertexts.
- *Related-key attack* – attacker can observe cipher operation with different unknown keys with known mathematical correlation between them.

Nowadays, the most well-know cryptanalysis methods are: linear, differential and brute-force [4].

- **Linear cryptanalysis**

This method was discovered and invented by Mitsuru Matsui and used to attack on FEAL and DES. The purpose of Linear Cryptanalysis is to find *linear approximation*, which describe function of a block cipher. For a given cipher algorithm:

$$P[i_1, i_2, \dots, i_a] \oplus C[j_1, j_2, \dots, j_b] = K[k_1, k_2, \dots, k_c]$$

where $i_1, i_2, \dots, i_a$, $j_1, j_2, \dots, j_b$ and $k_1, k_2, \dots, k_c$ denote bit location, and equation holds with probability $p \neq 0,5$ for pairs: plaintext P and ciphertext C . The magnitude of $|p - 0,5|$ represents the efficiency of linear approximation. In other words, if we analyze a sufficient number of plaintext–ciphertext pairs, we reveal the values for key bits (if $p \neq 0,5$).

Now, let us consider an example. The two central points of the attack are 14-rounds linear equations found by Matsui:

$$\begin{aligned}
 1) \quad & P_L^{(7)} \oplus P_L^{(18)} \oplus P_L^{(24)} \oplus C_H^{(7)} \oplus C_H^{(18)} \oplus C_H^{(24)} \oplus C_H^{(29)} \oplus C_L^{(15)} = \\
 & K_2^{(22)} \oplus K_3^{(44)} \oplus K_4^{(22)} \oplus K_6^{(22)} \oplus K_7^{(44)} \oplus K_8^{(22)} \oplus K_{10}^{(22)} \oplus K_{11}^{(44)} \oplus K_{12}^{(22)} \oplus K_{14}^{(22)} \\
 2) \quad & C_L^{(7)} \oplus C_L^{(18)} \oplus C_L^{(24)} \oplus P_H^{(7)} \oplus P_H^{(18)} \oplus P_H^{(24)} \oplus P_H^{(29)} \oplus P_L^{(15)} = \\
 & K_{13}^{(22)} \oplus K_{12}^{(44)} \oplus K_{11}^{(22)} \oplus K_9^{(22)} \oplus K_8^{(44)} \oplus K_7^{(22)} \oplus K_5^{(22)} \oplus K_4^{(44)} \oplus K_3^{(22)} \oplus K_1^{(22)}
 \end{aligned}$$

Where P_L is the lower 32 bit part of the 64 bit plaintext block, P_H the higher 32 bit part; C_L and C_H are respective parts of ciphertext; K_i – is the i-th DES sub-key.

The probability $p = 0,5 - 1,19 \cdot 2^{-21}$. So $\epsilon = 1,19 \cdot 2^{-21}$. It is small but we have 2^{47} plaintexts which give us 97,7% of success probability with 2^{42} DES evaluations. This attack allows us to find 26 bits of key, the rest must be found by exhaustive search. Full analysis of this attack can be found in [5].

• Differential cryptanalysis

This method was first introduced by Biham and Shamir in 1990. Differential cryptanalysis exploits the high probability of plaintext differences and differences in the last round of the cipher. For example, consider such an algorithm, where plaintext is $P = [P_1, P_2, \dots, P_n]$ and output $C = [C_1, C_2, \dots, C_n]$. Let two plaintexts to the system be P' and P'' with the corresponding ciphertexts C' and C'' . Input difference is given by $\Delta P = P' \oplus P''$ and the n-bits vector is:

$$\Delta P = [\Delta P_1, \Delta P_2, \dots, \Delta P_n]$$

And analogical for ciphertext C:

$$\Delta C = [\Delta C_1, \Delta C_2, \dots, \Delta C_n]$$

Differential cryptanalysis is a chosen plaintext attack. In this case, the attacker will select properly pairs of P' and P'' so, that the knowledge of ΔP value allows to determine value of ΔC with high probability.

Let us take one of DES S-boxes (S_5) which is presented in the Table 7.1.

Table 7.1: S-Box used in DES (S_5)

		Middle 4 bits of input															
Outer bits	S_5	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
	00	0010	1100	0100	0001	0111	1010	1011	0110	1000	0101	0011	1111	1101	0000	1110	1001
	01	1110	1011	0010	1100	0100	0111	1101	0001	0101	0000	1111	1010	0011	1001	1000	0110
	10	0100	0010	0001	1011	1010	1101	0111	1000	1111	1001	1100	0101	0110	0011	0000	1110
	11	1011	1000	1100	0111	0001	1110	0010	1101	0110	1111	0000	1001	1010	0100	0101	0011

Now we can define input XORs as IX and output XORs as OX. One IX can be achieved in many ways. For example: IX = 000100 can be achieved by: $110101 \oplus 110001$ or $000111 \oplus 000011$ and many more. For IX exists 64 ways. OX are 4 bit instead of 6 like IX. So for each OX we have 16 ways. Now let us construct special table: Rows $\rightarrow$ IX, Columns $\rightarrow$ OX and values inside represent how many pairs (IX,OX) we have.

Let us consider an example: IX: $001001 = 011011 \oplus 010010$ (only one from 64 combinations and we must test them all). Now we take S-box value for 011011 and for 010010:

$$S_5(011011) = 1001 \text{ (showed by yellow square)} \text{ and } S_5(010010) = 0101$$

Now we have: $OX = 1001 \oplus 0101 = 1100$,

So the (001001,1100) pair exists. Now we find proper element in table and increment it. We test all of 1024 ($64 \cdot 16$) (IX,OX) pairs 64 times.

The XOR table for S_5 is presented in the Table 7.2.

Table 7.2: XOR table for S_5

IX\OX	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	64	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
01	0	0	0	4	0	10	8	6	0	4	2	2	12	10	2	4
02	0	0	0	4	0	10	6	4	0	6	4	2	4	8	10	6
03	8	2	4	6	4	4	2	2	6	8	6	4	4	0	2	2
04	0	0	0	8	0	4	10	6	0	6	6	4	8	6	0	6
05	12	2	0	4	0	4	8	2	4	0	16	2	0	2	0	8
06	0	8	4	6	4	6	2	2	4	4	6	0	6	0	2	10
07	2	0	4	8	4	2	6	6	2	8	6	2	2	0	6	6
08	0	0	0	2	0	8	10	4	0	4	10	4	8	4	4	6
09	8	6	0	4	0	6	6	2	2	10	2	8	6	2	0	2
0A	0	6	8	6	0	8	0	0	8	10	4	2	8	0	0	4
0B	4	2	2	4	8	10	6	4	2	6	2	2	6	2	2	2
0C	0	0	0	10	0	2	10	2	0	6	10	6	6	6	2	4
0D	10	4	2	2	0	6	16	0	0	2	10	2	2	4	0	4
0E	0	6	4	8	4	6	10	2	4	4	4	2	4	0	2	4

0F	4	4	0	8	0	2	0	2	8	2	4	2	8	4	4	12
10	0	0	0	0	0	4	4	12	0	2	8	10	4	6	12	2
11	6	6	10	10	4	0	2	6	2	4	0	6	2	4	2	0
12	0	2	4	2	10	4	0	10	8	6	0	6	0	6	6	0
13	0	0	6	2	8	0	0	4	4	6	2	8	2	8	10	4
14	0	12	2	6	4	0	4	4	8	4	4	4	6	2	4	0
15	4	8	0	2	8	0	2	4	2	2	4	2	4	8	8	6
16	0	6	10	2	14	0	2	2	4	4	0	6	0	4	6	4
17	0	6	8	4	8	4	0	2	8	4	0	2	2	8	6	2
18	0	10	8	0	6	4	0	4	4	4	6	4	4	4	0	6
19	0	4	6	2	4	4	2	6	4	2	2	4	12	2	10	0
1A	0	2	16	2	12	2	0	6	4	0	0	4	0	4	4	8
1B	2	8	12	0	0	2	2	6	8	4	0	6	0	0	8	6
1C	0	10	2	6	6	6	6	4	8	2	0	4	4	4	2	0
1D	4	6	2	0	8	2	4	6	6	0	8	6	2	4	2	4
1E	0	2	6	2	4	0	0	2	12	2	2	6	2	10	10	4
1F	0	6	8	4	8	8	0	6	6	2	0	6	0	6	2	2
20	0	0	0	8	0	8	2	6	0	4	4	4	6	6	8	8
21	0	0	0	6	6	2	6	4	6	10	14	4	0	0	4	2
22	14	4	0	10	0	2	12	2	2	2	10	2	0	0	2	2
23	2	0	0	4	2	2	10	4	0	8	8	2	6	8	0	8
24	6	2	8	4	4	4	6	2	2	6	6	2	6	2	2	2
25	6	0	0	8	2	8	2	6	6	4	2	2	4	2	6	6
26	12	0	0	4	0	4	4	4	0	8	4	0	12	8	0	4
27	12	2	0	2	0	12	2	2	4	4	8	4	8	2	2	0
28	2	8	4	6	2	4	6	0	6	6	4	0	2	2	2	10
29	6	4	6	8	8	4	6	2	0	0	2	2	10	0	2	4
2A	4	4	0	2	2	4	6	2	0	0	6	4	10	4	4	12
2B	4	6	2	6	0	0	12	2	0	4	12	2	6	4	0	4
2C	8	6	2	6	4	8	6	0	4	4	0	2	6	0	6	2
2D	4	4	0	4	0	6	4	2	4	12	0	4	4	6	4	6
2E	6	0	2	4	0	6	6	4	2	10	6	10	6	2	0	0
2F	10	4	0	2	2	6	10	2	0	2	2	4	6	2	2	10
30	0	4	8	4	6	4	0	6	10	4	2	4	2	6	4	0
31	0	6	6	4	10	2	0	0	4	4	0	0	4	6	12	6
32	4	6	0	2	6	4	6	0	6	0	4	6	4	10	6	0
33	8	10	0	14	8	0	0	8	2	0	2	4	0	4	4	0
34	0	4	4	2	14	4	0	8	6	8	2	2	0	4	6	0
35	0	4	16	0	8	4	0	4	4	4	0	8	0	4	4	4
36	4	4	4	6	2	2	2	12	2	4	4	8	2	4	4	0
37	4	2	2	2	4	2	0	8	2	2	2	12	6	2	8	6
38	0	4	8	4	12	0	0	8	10	2	0	0	0	4	2	10
39	0	8	12	0	2	2	2	2	12	4	0	8	0	4	4	4
3A	0	14	4	0	4	6	0	0	6	2	10	8	0	0	4	6
3B	0	2	2	2	4	4	8	6	8	2	2	2	6	14	2	0
3C	0	0	10	2	6	0	0	2	6	2	2	10	2	4	10	8
3D	0	6	12	2	4	8	0	8	8	2	2	0	2	2	4	4
3E	4	4	10	0	2	4	8	8	2	2	0	2	6	8	4	0
3F	8	6	6	0	4	2	2	4	4	2	8	6	2	4	6	0

As we see in the Table 7.2, some pairs are more common (like (1A,02)) than others and some do not exist (have 0 in their matrix value). Thanks to that we can (by choosing special 2^{14} plaintexts) break 8 round DES in few minutes on home PC. Full 16 rounds is still hard to break by differential cryptanalysis (2^{47} plaintexts required) in short time.

- **Brute-force**

This simple method is very popular. Brute-force is nothing other than trying all possible keys. Unfortunately (for the attacker) number of keys is huge: 2^n , where n is a length of key. For example, in DES ciphertext has to be decrypted and tested for about $72 \cdot 10^{15}$ keys to produce true key. Only way to avoid brute-force attack is “appropriate” length of key. “Appropriate” key length is one, that ensures that the time needed to break the cipher is greater than the time that makes ciphered content meaningless or irrelevant.

Now let us consider the cryptanalysis of new cipher. In new cipher all of transformations are highly nonlinear (112) so there is no way to create linear equations and have only 0, 2 and one 4 in XOR matrix.

Main flaw is that half of the S-boxes have mapping $52_{(HEX)} \rightarrow 00_{(HEX)}$ and other half $63_{(HEX)} \rightarrow 00_{(HEX)}$. Let us consider the least safe option 128bit key length and 8 rounds. It uses only 2 S-boxes: one to substitute and one to permute.

Now let us look closer to 3 chosen 256 bit plaintexts (in HEX format):

Pa: 52...

Pb: 63...

Pc: 00...

Now let us denote: $Ca(1)$ – ciphered plaintext Pa after 1st round, $Ca(2)$ – after 2nd round and so on, corresponding $Cb(\#), Cc(\#)$.

We have $p = 0,5$ that substitution S-box will have $52 \rightarrow 00$ flaw. When it happens after substitution in first round with S-box every 52 in plaintext Pa will become only zeroes that is Pc. For other $q = 0,5$ substitution S-box will have $63 \rightarrow 00$ flaw. For Pb after substitution with that kind of S-box we have only zeroes. Permutation is irrelevant here because there are only zeroes.

Half of the time after first round plaintext Pa will become Pc and other half of the time plaintext Pb after 1st round will become Pc. We can write:

With $p = 0,5$:

$$Ca(1) = Pc$$

and with $q = 1 - p = 0,5$:

$$Cb(1) = Pc$$

After n rounds we have with $p = 0,5$:

$$Ca(n) = Cc(n - 1)$$

and with $q = 1 - p = 0,5$:

$$Cb(n) = Cc(n - 1)$$

So summarizing after n rounds:

$$Pa \rightarrow Cc(n - 1) \text{ with } p = 0,5$$

$$Pb \rightarrow Cc(n - 1) \text{ with } q = 0,5$$

$$Pc \rightarrow Cc(n)$$

In Figure 7.1 we have presented some scheme (we have s-box with $52 \rightarrow 00$ flaw and Pa and Pc). At the end from Pa we have $Ca(n) = Cc(n - 1)$ and $Cc(n) = Ca(n + 1)$. So instead of n rounds we only have one round to analyze with brute force. We can eliminate it by adding permutation at the beginning of the first round. It will “shuffle” characteristic plaintexts so we won't have only zeroes after 1st round.

Weak keys are present in the new cipher but only in the 192, 320 and 576 key lengths. The main problem is that weak key grants the same round encryption as shorter key. Every 64 bit block of the key represents one s-box. At 192 we have 3 s-boxes, at 320 – 5, and at 576 – 9. The last s-box is used to generate permutation. Let us mark 64 bit parts of the key as “A”, “B”, “C” and so on.

Now we give examples (scheme in the Figure 7.2 covers an example ‘a’):

- a) 576 bit key “AAAAAAAAB” equals 320 bit key “AAAAB” equals 192 bit key “AAB” equals 128 bit key “AB”,
- b) 576 bit key “AAAABBBBC” equals 320 bit key “AABBC” equals 192 bit key “ABC”,
- c) 576 bit key “AABBDDCCE” equals 320 bit key “ABCDE”.

So we see that those keys are worse because they give the same substitution as S-boxes generated from shorter keys. Of course if we use longer key lengths we have more rounds, so the whole encryption differs.

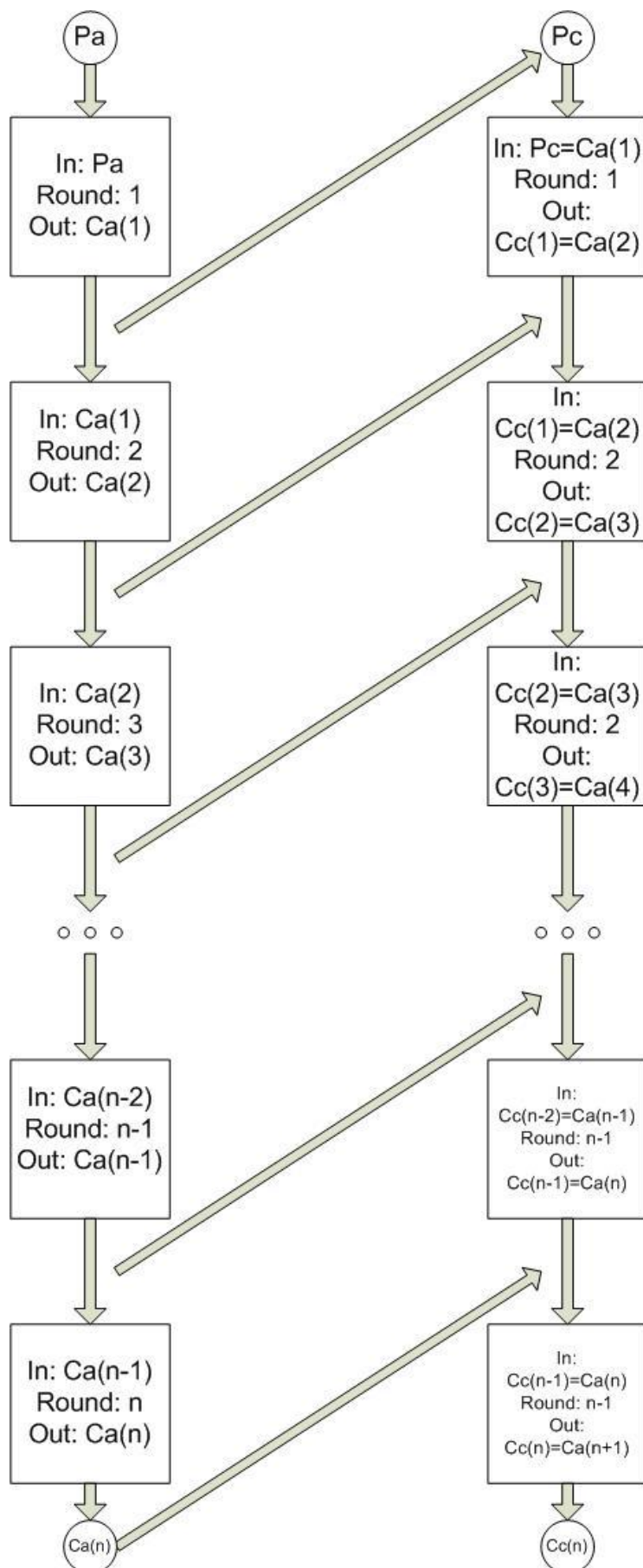


Figure 7.1: The scheme: s-box with 52 → 00 flaw, Pa , and Pc

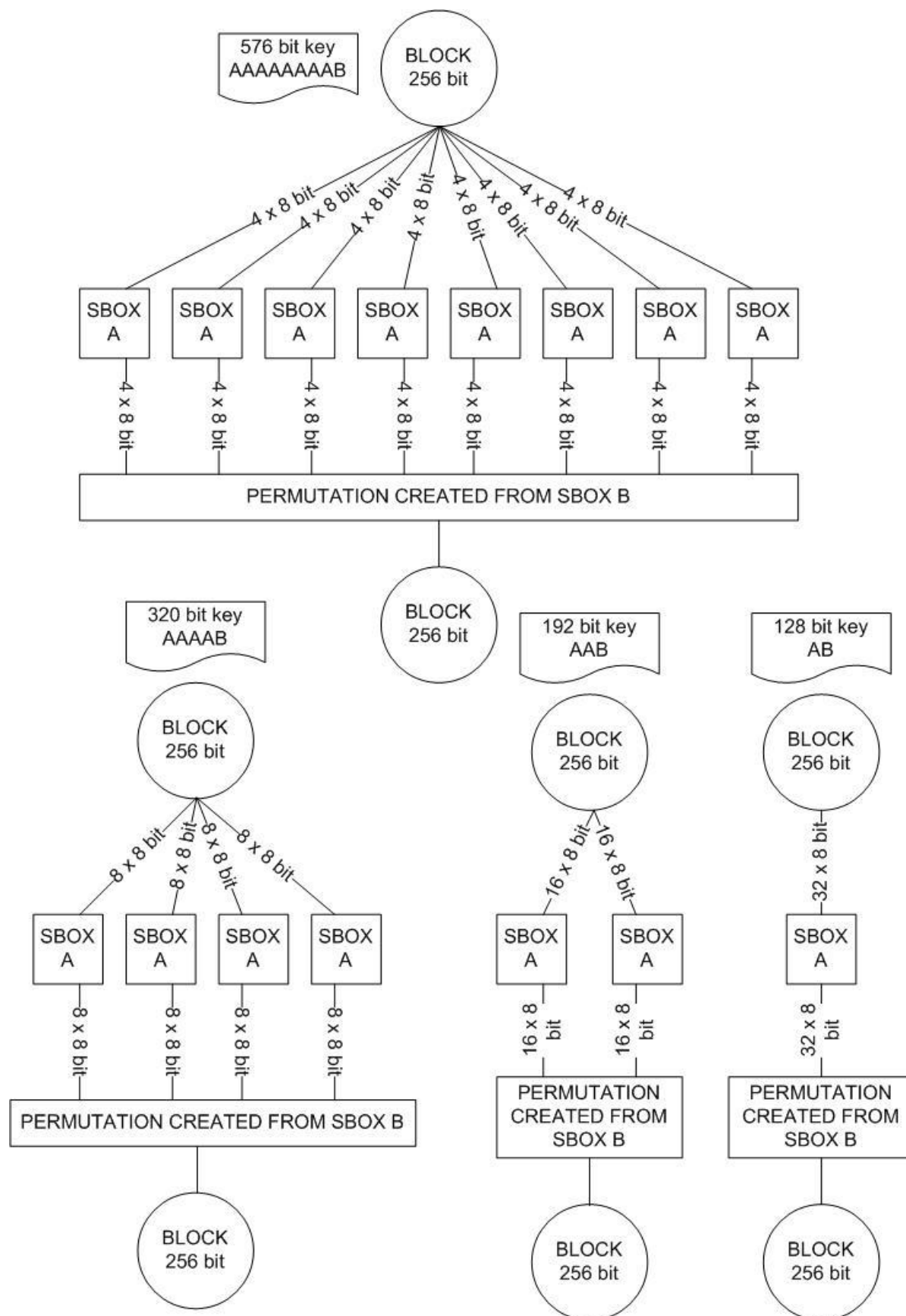


Figure 7.2: The scheme with 576 bit key "AAAAAAAAB"

Now let us look how many weaker keys exist: We have approximately 10^{19} S-boxes, let us mark this number as “SN”. There are:

- SN^2 128 bit keys with no weaker keys
- SN^3 192 bit keys which contains SN^2 weaker keys ($1/SN$ probability)
- SN^5 320 bit keys which contains SN^3 weaker keys ($1/SN^2$ probability)
- SN^9 576 bit keys which contains SN^5 weaker keys ($1/SN^4$ probability)

The number of weaker keys represents those keys which have at least one equivalent in shorter keys. Probability to generate weaker key is very low so before the implementation we can choose two ways:

- we can omit the fact of their existence because still number of rounds is different for each key length
- or we can change the key generator to rule out those keys

7.2 The simulator of block ciphers

The Simulator is a tool, used to check some crucial properties of symmetric ciphers. These properties have direct impact on the security level of the algorithm. Functions, which are implemented in simulator are described in chapter 7.3 in details. Below, we present main building blocks of simulator application as well as some examples of performed simulations.

The simulator is written in the C++ language using the Microsoft Visual C++ 2008 integrated development environment. The whole source code of the simulator is divided into four parts:

- Binary operations — are required because application works on strings. In this part of source code are used functions, which convert numeral systems, data types, etc.
- Input and output functions — these functions are responsible for importing and processing input data as well as for displaying the results.
- Design criteria — in this part of source code functions described in chapter 7.3 are defined.
- Main — contains the executable part of the simulator and console interface.

Adding new functions to the simulator is relatively easy. Appropriate function should be added to Design criteria block, and then also placed in the main() function for execution. New functions may operate on input bits, output bits, XOR table and other objects (new objects should be added to Input and output functions).

Now let us consider an example of simulation:

1. Before performing the simulation, user must define cipher by creating arrays of inputs and outputs or use implemented ciphers. Binary values, which describe whole tested algorithm (every possible combinations of

input and output bit chains) should be placed in input and output arrays, as shown in Figure 7.3. These files must be named input.txt and output.txt.

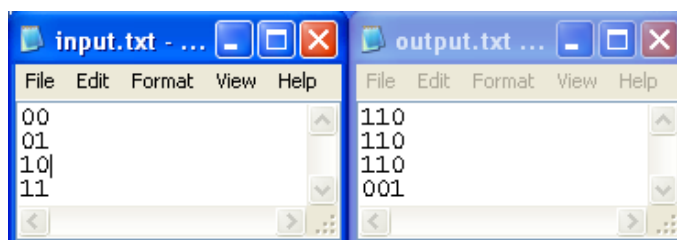


Figure 7.3: Input and output file

If user employs the implemented ciphers, the application requires a file with the substitution box (S-box). Depending on the cipher S-boxes must have a different structure and name. Figure 7.4 shows correct structure of S-boxes for AES and DES ciphers.

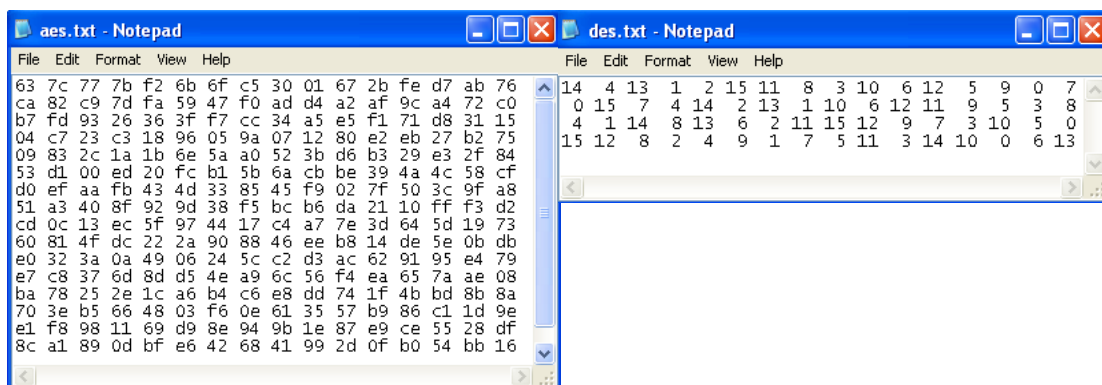


Figure 7.4: Examples of S-boxes

2. The user who opens the simulator is prompted to choose what he/she wants to test. The options are:
 - DES S-box (des.txt is required)
 - AES S-box (aes.txt is required)
 - Custom input and output tables (input.txt and output.txt are required)
 - Static/Dynamic criteria (any text file named input_file.txt is needed)

All input files should be in the same folder as application.

3. If the user chooses one of the first three methods, he/she is prompted to determine whether the results should be saved to file (results.txt), or displayed on the screen, as shown in Figure 7.5. Obtained results describe tested features such as:

- Balancing
- Nonlinearity
- SAC
- Completeness
- Diffusion order
- Lox XOR table

These features are described in greater detail in Chapter 7.3.

```
Select what you want to simulate
[1] DES S-box
[2] AES S-box
[3] Custom output table
[4] Static/Dynamic IO/OI criteria
2
Do you want save results to file?[y/n]
n
AES S-box checking...
63 7c 77 7b f2 6b 6f c5 30 01 67 2b fe d7 ab 76
ca 82 c9 7d fa 59 47 f0 ad d4 a2 af 9c a4 72 c0
b7 fd 93 26 36 3f f7 cc 34 a5 e5 f1 71 d8 31 15
04 c7 23 c3 18 96 05 9a 07 12 80 e2 eb 27 b2 75
09 83 2c 1a 1b 6e 5a a0 52 3b d6 b3 29 e3 2f 84
53 d1 00 ed 20 fc b1 5b 6a cb be 39 4a 4c 58 cf
d0 ef aa fb 43 4d 33 85 45 f9 02 7f 50 3c 9f a8
51 a3 40 8f 92 9d 38 f5 bc b6 da 21 10 ff f3 d2
cd 0c 13 ec 5f 97 44 17 c4 a7 7e 3d 64 5d 19 73
60 81 4f dc 22 2a 90 88 46 ee b8 14 de 5e 0b db
e0 32 3a 0a 49 06 24 5c c2 d3 ac 62 91 95 e4 79
e7 c8 37 6d 8d d5 4e a9 6c 56 f4 ea 65 7a ae 08
ba 78 25 2e 1c a6 b4 c6 e8 dd 74 1f 4b bd 8b 8a
70 3e b5 66 48 03 f6 0e 61 35 57 b9 86 c1 1d 9e
e1 f8 98 11 69 d9 8e 94 9b 1e 87 e9 ce 55 28 df
8c a1 89 0d bf e6 42 68 41 99 2d 0f b0 54 bb 16

Probability of changing output bit if one input bit is changed(SAC) equals: 50%
Completeness is ensured for 100% possible inputs
faulty XOR distribution
diffusion order equals: 0
function is balanced
minimum distance to affine function equals (bent function has 128):
for 1 bit 112      (most significant)
for 2 bit 112
for 3 bit 112
for 4 bit 112
for 5 bit 112
for 6 bit 112
for 7 bit 112
for 8 bit 112
```

Figure 7.5: Example of Simulator results for AES S-box testing

4. If user chooses the fourth option (Static/Dynamic IO/OI criteria) another menu is shown where one can further choose between functions, described in greater detail in chapter 7.3, such as:

- **Invert S-box**

Inverts S-box stored in aes.txt file and saves the results in iverted_sbox.txt file as well as displays it on the screen

- **File to S-box**

Processes earlier prepared input_file.txt file through S-box stored in aes.txt file. Results are saved in output_file.txt

- **Static I/O**

Checks the static I/O criteria based on the hexadecimal value provided by the user. It works on files: input_file.txt and output_file.txt obtained with File to S-box function.

- **Static O/I**

Checks the static O/I criteria based on the hexadecimal value provided by the user. It works on files: input_file.txt and output_file.txt obtained with File to S-box function.

- **Dynamic I/O**

Checks the dynamic I/O criteria based on the hexadecimal values provided by the user. It works on two pairs of files: input_file.txt and output_file.txt as well as input_file_2.txt and output_file_2.txt. Both pairs should be obtained with File to S-box function.

- **Dynamic O/I**

Checks the dynamic O/I criteria based on the hexadecimal values provided by the user. It works on two pairs of files: input_file.txt and output_file.txt as well as input_file_2.txt and output_file_2.txt. Both pairs should be obtained with File to S-box function.

The second menu is shown in Figure 7.6.

```
Select what you want to simulate
[1] DES S-box
[2] AES S-box
[3] Custom output table
[4] Static/Dynamic IO/OI criteria
4
Select criteria/function
[1] Invert S-box
[2] FSB0X
[3] Static IO
[4] Static OI
[5] Dynamic IO
[6] Dynamic OI
```

Figure 7.6: The second menu of the simulator.

An example simulation is presented in Figure 7.7.

```

Select criteria/function
[1] Invert S-box
[2] FSB0X
[3] Static IO
[4] Static OI
[5] Dynamic IO
[6] Dynamic OI
5
Insert hexadecimal value you want to check:
70
Files 1 <input_file.txt and output_file.txt>
Count x1:2
Invert x1:51
Count y1:2

Insert hexadecimal value for another pair of files:
72
Files 2 <input_file_2.txt and output_file_2.txt>
Count x2:1
Invert x2:40
Count y2:1
Delta x=x1-x2=1
Delta y=y1-y2=1

```

Figure 7.7: Example of simulation of Dynamic I/O criteria

In the section 6.3 we mentioned that we are able to create 5348063769211699200 (about $5,35 \cdot 10^{18}$) S-boxes. We proved that all of them have the same security level as S-box used in the AES cipher (nonlinearity=112, etc.). It is not possible to test all of them with use of the simulator but we checked some of them. During a few weeks we tested over 43 million S-boxes. All tested S-boxes have the same security level as S-box used in the AES cipher.

7.3 Tested features

In this section we present all security features which could be tested in the new simulator. Some of these functionalities were described in section 4.2. Therefore in this section we present only general information and source code. The rest of functionalities are described in detail.

- **Balancing**

This feature ensures that S-boxes do not discriminate against any of the bits, so no value is favoured. In this case it is sufficient to check the number of “1”. If function is balanced the number of “1” is half of all possible output bits. The source code of this function is presented in Listing 7.1.

Listing 7.1: Source code of balancing function

```

void balance (int inlength, int outlength, string* out)
{
    int numof1=0, d1;
    d1=pow2(inlength);
    for (int i=0; i<d1; i++)
        numof1=wt(out[i])+numof1;
}

```

```

        if (numof1==(outlength*d1/2))
            cout<<"function is balanced"<<endl;
        else
            cout<<"function is not balanced"<<endl;
    }

```

- **The Strict Avalanche Criterion (SAC)**

It is satisfied if the change of a single input bit changes each of the output bits with a probability of one half. The source code of this function is presented in Listing 7.2.

Listing 7.2: Source code of SAC function

```

void SAC (int inlength, int outlength, string* out)
{
    int i, j, proball, change=1;
    for (i=1; i<pow2(inlength); i++)
    {
        for (j=0; j<outlength; j++)
        {
            if (out[i][j] != out[i-1][j])
                change=change+1;
        }
    }
    proball = (100*change)/(i*outlength);
    cout<<"Probability of changing output bit if one input ";
    cout<<"bit is changed(SAC) equals: "<<proball<<"%"<<endl;
}

```

- **Completeness**

Completeness ensures that for each selected entry, we can choose the value of other inputs in such way that it will bring change to the entry of any change in output. The source code of the first part of completeness function (the second part deals with inserting "0") is presented in Listing 7.3.

Listing 7.3: Source code of completeness function

```

void complet (string *out, int inlength, int outlength)
{
    string out2, re, re2, prev;
    int i=0, j=0, k, sum=0, sum2=0, l, completeness;
    for (l=0; l<inlength; l++)
    {
        j=0;
        for (i=0; i<outlength; i++)

```

```

{
    for (k=0; k<pow2(inlength-1); k++)
    {
        re=dec2bin(inlength-1,k);
        re2= re.insert(1,"1");
        out2 = out[bin2dec(re2)];
        if (k==0)
            prev=out2;
        if (out2[i]!=prev[i])
        {
            j++;
            break;
        }
        prev = out2;
    }
    sum=sum+j;
    sum2=sum2+i;
}
completeness=100*sum/sum2;
cout<<"Completeness is ensured for "<<completeness;
cout<<"%"<<" possible inputs"<<endl;
}

```

- **Diffusion Order**

It ensures that, even if the value of the output bits change is large, the number of changes in the entry is relatively low. Formally:

$wt(dY) > \lambda + 1 - wt(dX)$ for $wt(dX) < \lambda + 1$, where λ is a diffusion order.

The source code of this function is presented in Listing 7.4.

Listing 7.4: Source code of diffusion order function

```

void diffusion (int inlength, int outlength, int **XOR)
{
    int d1=pow2(inlength),d2=pow2(outlength);
    int wtXORin,wtXORout,stop=0,L,b;
    string s;
    for (L=0;;L++)
    {

```

```

b=0;
for (int i=1; i<d1; i++)
{
    wtXORin=wt(dec2bin(inlength,i));
    for (int j=0; j<d2; j++)
    {
        if ((XOR[i][j])!=0)
        {
            s=dec2bin(outlength,j);
            wtXORout=wt(s);
            if ((wtXORin<(L+1)) && (wtXORout<=L+1-wtXORin))
                stop=1;
        }
    }
    if (stop==1)
        break;
}
L=L-1;
cout<<"diffusion order equals: "<<L<<endl;
}

```

- **Low XOR Table**

S-box is secure when has low XOR table entries, ideally — “0” or “2”. The source code of this function is presented in Listing 7.5.

Listing 7.5: Source code of checking XOR function

```

void chkxor (int inlength, int outlength, int **XOR)
{
    int d1=pow2(inlength),d2=pow2(outlength);
    int c1,c2,c3,zeros,twos;
    for (int i=0; i<d1;i++)
    {
        twos=0;zeros=0;
        for (int j=0;j<d2;j++)
        {
            if (i==0 && j==0)
                // checking [0,0] value in XOR table

```

```

    {
        if (XOR[i][j]==d1)
        {
            c1=1;
            c2=1;
        }
    }
    if(i==0)
    // checking first row in xor table
        if (XOR[i][j]==0)
            c1=c1+1;

    if (j==0)
    // checking first column in xor table
        if (XOR[i][j]==0)
            c2=c2+1;

    if (XOR[i][j]==0 && i!=0 )
    //checking rest of xor table
        zeros=zeros+1;

    if (XOR[i][j]==2 && i!=0)
        twos=twos+1;

    if (twos==zeros && zeros==(d2/2))
        c3=1;
    }
}

if (c1==d2 && c2==d1 && c3==1)
    cout<< "ideal XOR distribution"<<endl;
else
    cout<< "faulty XOR distribution"<<endl;
}

```

• Nonlinearity

Function calculating affine and function truth table (from inputs and outputs table). Hamming distance (and unexpected Hamming distance) is calculated from the truth table. The Hamming distance determines nonlinearity. T. Ritter described this method in Measuring Boolean Function Nonlinearity by Walsh Transform. The source code of this function is presented in Listing 7.6.

Listing 7.6: Source code of nonlinearity function

```
void nonlinear (int inlength, int outlength)
{
    int a=0, up, d2, outbit, mindist, maxdist, undist, distance;
    string f;
    d2=pow2(inlength);
    int* dist = new int [d2];
    string* affine = new string [d2];
    string* truthtable = new string [d2];

    affine[0]=i2b(inlength,0);
    for (int i=1; i<d2; i++)
    {
        affine[i]=dec2bin(inlength,i);
    }

    for (int i=0; i<d2; i++) //Truth Table for affine function
        truthtable[0]=truthtable[0].insert(i,"1");
    for (int i=1; i<d2; i++)
    {
        for (int j=0; j<d2; j++)
        {
            a=0;
            for (int k=0; k<inlength; k++)
            {
                if (affine[i][k]=='1')
                {
                    up=static_cast<int>(in[j][k])-48;
                    a= up xor a;
                }
            }
            truthtable[i]=
                truthtable[i].insert(j,(dec2bin(1,a)));
        }
    }

    cout<<"minimum distance to affine function equals ";
    cout<<"(bent function has "<<d2/2<<"): "<<endl;
    for (int k=0; k<outlength; k++) //Truth Table for S-box
        function
    {
        mindist=0;
        f.clear();
        for (int i=0; i<d2; i++)
        {
            outbit=static_cast<int>(out[i][k])-48;
            f=f.insert(i,dec2bin(1,outbit));
        }
    }
}
```

```

for (int i=0; i<d2; i++)          //Hamming distance
{
    dist[i]=wt(sXOR(truthtable[i],f));
    if (i!=0)
    {
        if(mindist>dist[i])
        {
            mindist=dist[i];
        }
    }
    else
        mindist=dist[i];
    if (i!=0)
    {
        if(maxdist<dist[i])
        {
            maxdist=dist[i];
        }
    }
    else
        maxdist=dist[i];
    if (mindist<d2-maxdist)
    {
        distance=mindist;
    }
    else
        distance=d2-maxdist;

    if (i!=0)
    {
        if(undist<abs(dist[i]-d2/2))
        {
            undist=abs(dist[i]-d2/2);
        }
    }
    else
        undist=abs(dist[i]-d2/2);
}
cout<<"for "<< k+1 <<" bit "<<distance<<" ";
if(k==0)
{
    cout<<" (most significant) ";
}
cout<<endl;
}

```

- **Function “Invert S-BOX”**

This is an auxiliary function which is designed to generate an inverted s-box based on the uploaded (input) one. In order to receive a reversed s-box, two cycles are created to read in sequence the original s-box values by rows and columns. The read

string's values are converted into upper case. The numeric value of the first symbol of the read string is assigned to the variable x (The string consists of two symbols) i.e. $x=s[1]$. The numeric value of the second symbol of the read string is assigned to the y variable (i.e. $y=s[2]$). The Function HexToInt() converts the Hex string value from 0 to F into numeric i.e. from 0 to 15. The values s for x and y are used as coordinate's value that has to be assigned to a massive of 16x16 dimensions, i.e. the current values of the cycle counters (i,j) are converted into Hex format and are assigned to a cell with coordinates j,i i.e. $cell[x,y]=Hex(ij)$. On Figure 7.8 the block scheme of the algorithm for the described function is shown. Listing 7.7 contains source code for this function.

Listing 7.7: Source code of invert Sbox function

```
void InverSbox(string* aesbox) {
    string s;
    string a;
    for(int i = 0; i < 16; i++){
        for(int j = 0; j < 16; j++){
            s = aesbox[i*16+j];
            int x = HexToInt(s[0]);
            int y = HexToInt(s[1]);
            a="";
            a+=IntToHex(i);
            a+=IntToHex(j);
            inverted_sbox[16*x+y]=a;
        }
    }
}
```

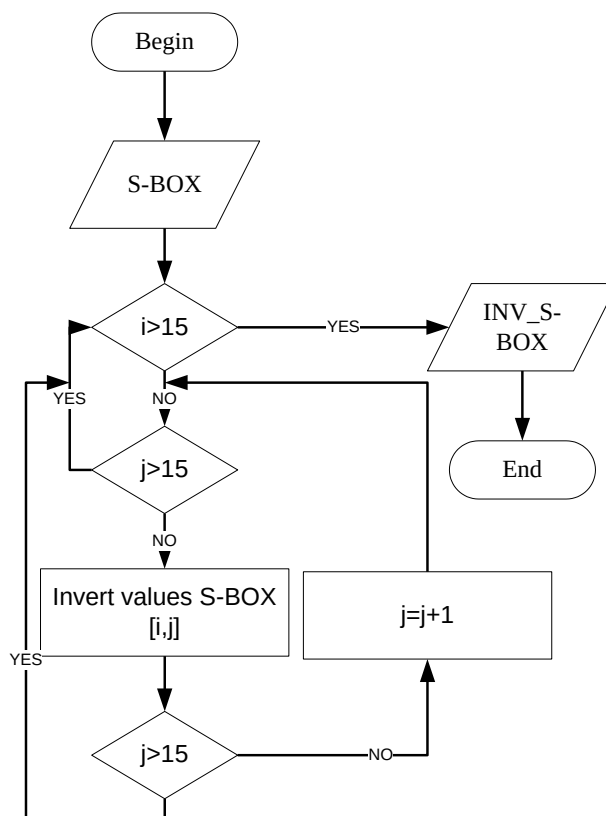


Figure 7.8: Invert Sbox function scheme

- **Function “File to S-BOX”**

Input parameters to this auxiliary function are the name of the file, containing the s-box and the name of the file which will be processed. The selected s-box is loaded into a matrix of 16x16, after that the input file is read from the beginning to the end byte by byte. For each read byte the following operations are conducted: The format of the read file is a string. Numeric values of the first and the second symbol of the string are assigned respectively to the two variables x and y. Next the value from the matrix with coordinates [x,y] is defined. The value from the matrix (based on the uploaded s-box) is converted into bytes and is written into new (input) file. In this way the operation is repeated till the end of the input file. The size of the input and output files is the same. The block scheme of the algorithm of the described function is shown in Figure 7.9. Listing 7.8 presents source code of this function.

Listing 7.8: Source code of File to S-box function

```

void FileToSbox(string* aesbox){
    unsigned char ch;
    int j;
    string s;
    string t;
    int i = 0;

```

```
FILE* f = fopen("input_file.txt", "rb");
FILE* f1 = fopen("output_file.txt", "wb");
fseek(f,0,SEEK_SET);
do
{
    ch = fgetc(f);
    if (!feof(f)){
        s = bin2hex(dec2bin(inlength, (int)ch));
        int x = HexToInt(s[0]);
        int y = HexToInt(s[1]);
        t = "0x" + aesbox[16*x+y];
        j=bin2dec(hex2bin(aesbox[16*x+y]));
        fwrite(&j, 1, 1, f1);
    }
}
while (!feof(f));          // ch != EOF
fclose(f);
fclose(f1);
}
```

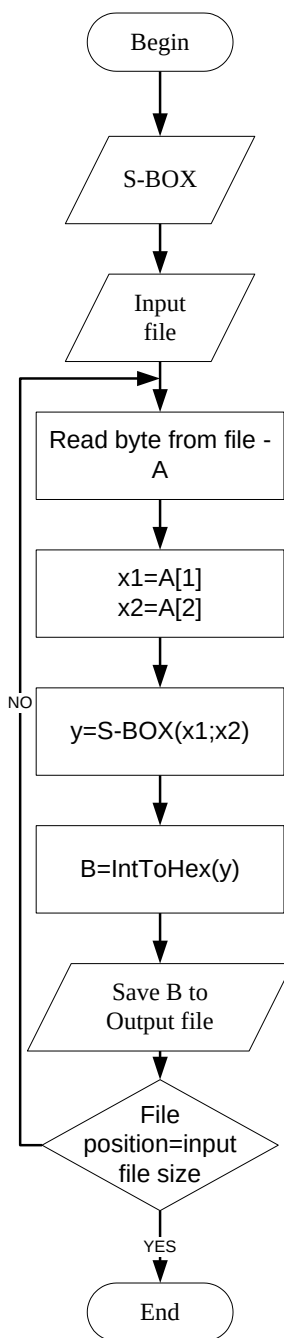


Figure 7.9: File to S-box function scheme.

- **Function “Stat I/O”**

The function „Stat I/O” checks the dependence (or independence) between input and output data of the given s-box. As input data for this function, the ones of the s-box in concern are used, the file (Original) received after processing with the function “File to S-BOX” with the used s-box and the same input (original) file, the Hex value of which is checked for input/output dependence. The selected s-box is loaded into a 16x16 matrix. The original file is read from the beginning to the end, counting the number of bytes with the given Hex value. Based on the selected s-box using the method of the functions described above, the inverted value of the processed byte

input in Hex format is assigned. The Binary File 2 is read from the beginning to the end, counting the number of appearances of the inverted value.

The result of this function is: The number of appearances of the given Hexadecimal value in the Binary file 1. The frequency of appearance of this value, calculated as: $P(y/x) = \text{number of appearance} / \text{size of file}$, invert value of the given file, the number of appearance of the invert byte in the Binary file 2, the frequency of appearance of the invert byte under the same formula. The results for number of appearance and the frequency have to be equal. The block-scheme of the algorithm of the function “Stat I/O” is shown in Figure 7.10.

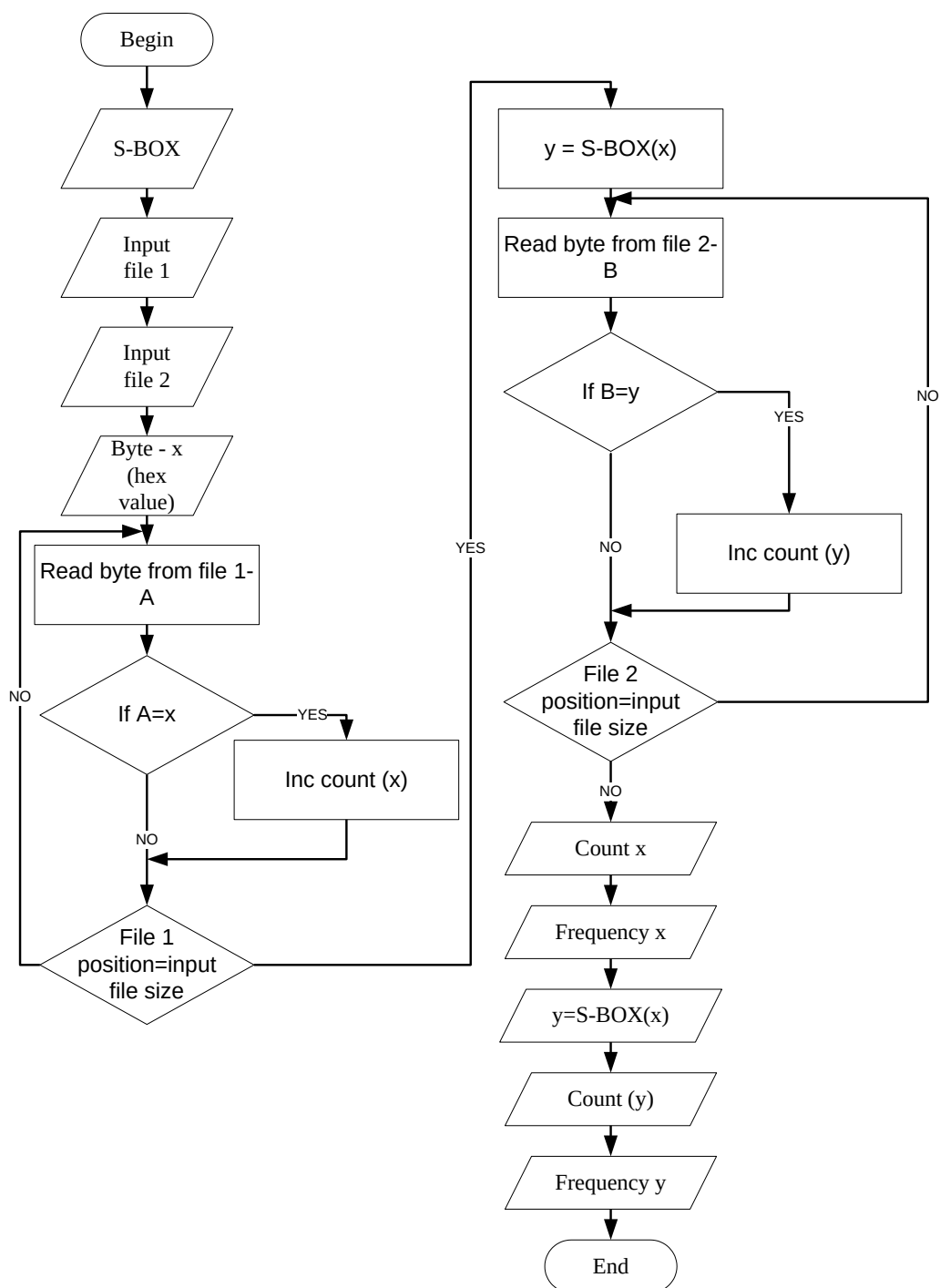


Figure 7.10: Stat I/O function scheme

- **Function “Stat O/I”**

The function „Stat O/I” checks the dependence/independence between input and output data for the given s-box. As input data for this function is the inverted s-box, the file (original), file is received after processing with the function “File to S-BOX” with the given s-box and the same input (original) file the Hexadecimal value of which will be checked for input/output dependence. The selected invert s-box is loaded into a matrix of size 16x16. The Binary File 2 is read from the beginning to the end counting the number of appearance of the given byte. Based on the used inverted s-box via the way described in the previous functions the value of the given byte entered into Hexadecimal format. The original file is read from the beginning to the end counting the number of appearance of bytes with the resulted Hexadecimal value of the byte.

The result of this function is: The number of appearance of the given Hexadecimal value in the Binary file 1. The frequency of appearance of this value, calculated as: $P(y/x) = \text{number of appearance} / \text{size of file}$, invert value of the given file, the number of appearance of the invert byte in the Binary file 2, the frequency of appearance of the invert byte under the same formula. The results for number of appearance and the frequency have to be equal. On Figure 7.11 the block-scheme of the algorithm of the function “Stat O/I” is shown.

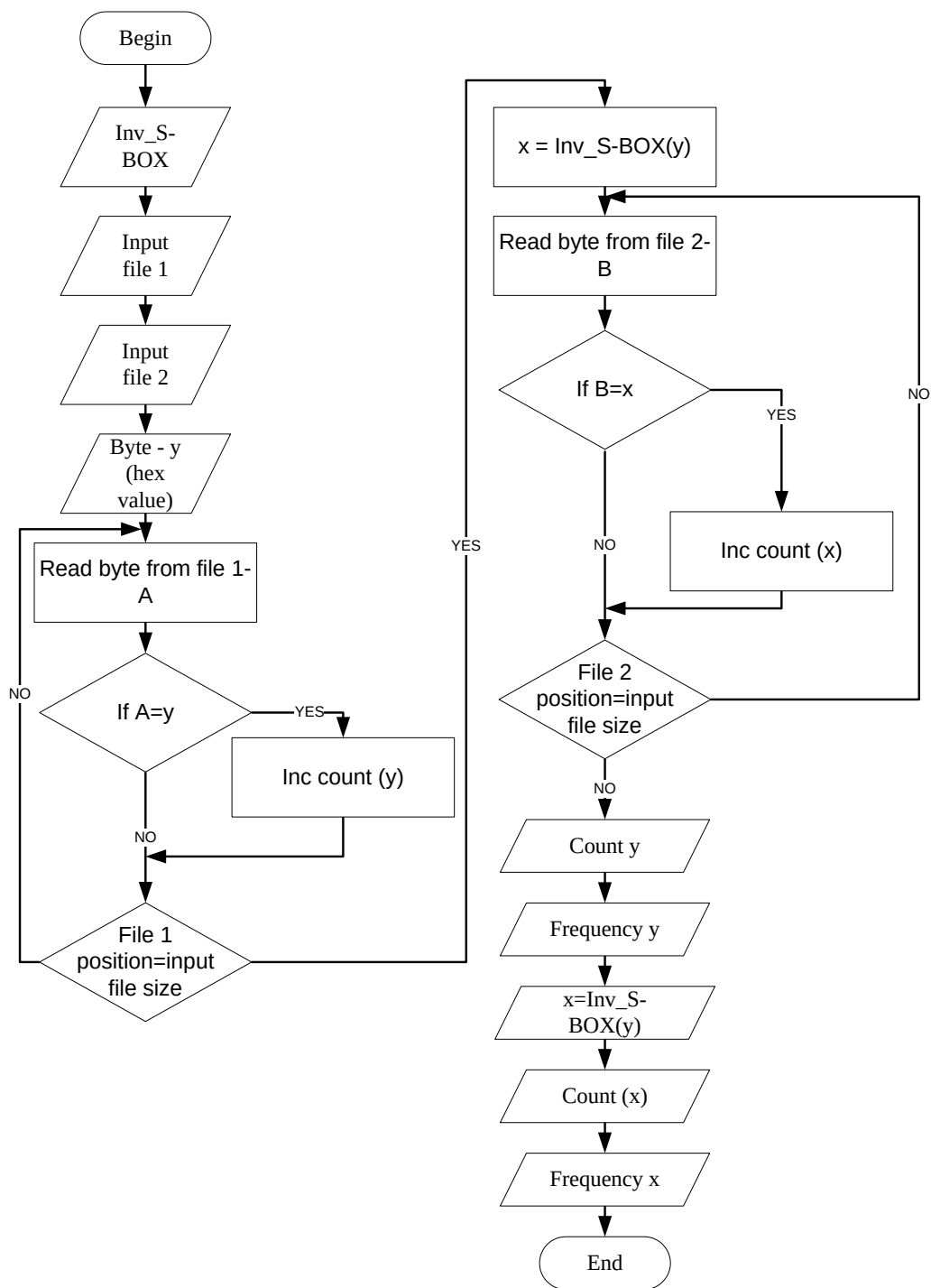


Figure 7.11: Stat O/I function scheme.

- **Function “Dynamic I/O”**

The function „Dynamic I/O” checks the dynamic dependence (independence) between input and output data for a given s-box. The input data for this function are: the given s-box being used, the files (source 1 and source 2) received after their processing with the “File to S-BOX” function with the given s-box and the same input original files, where the bytes will be checked for dynamic input/output dependence. The selected s-box is loaded into a matrix of size 16x16. The original file (source 1) is read from the beginning to the end, and the number of bytes with the given

Hexadecimal value (x1) are counted. Based on the given s-box, and using the methods described in the previous functions, the invert value of the checked byte (x1) introduced in Hexadecimal format (y1) is calculated. The Binary File (destination 1) is read from the beginning to the end, counting the number of appearances of the inverted value (y1). The described actions are repeated for the input file (source 2), output file (destination 2) and byte (x2).

The result of the function is: The number of appearances of the given Hexadecimal value (x1) in the Binary file (source 1), the inverted value of the given byte y1, the number of appearances of the inverted byte in the Binary file (destination 1), the number of appearances of the given Hexadecimal value (x2) in the Binary file (source 2), the inverted value of the given byte y2, the number of appearances of the inverted byte in the Binary file (destination 2). The differences ($\Delta x = x1 - x2$ and $\Delta y = y1 - y2$) are calculated.

The results for the appearances numbers and deltas have to be equal. On Figure 7.12 the block – scheme of the algorithm of the “Dynamic I/O” function is given.

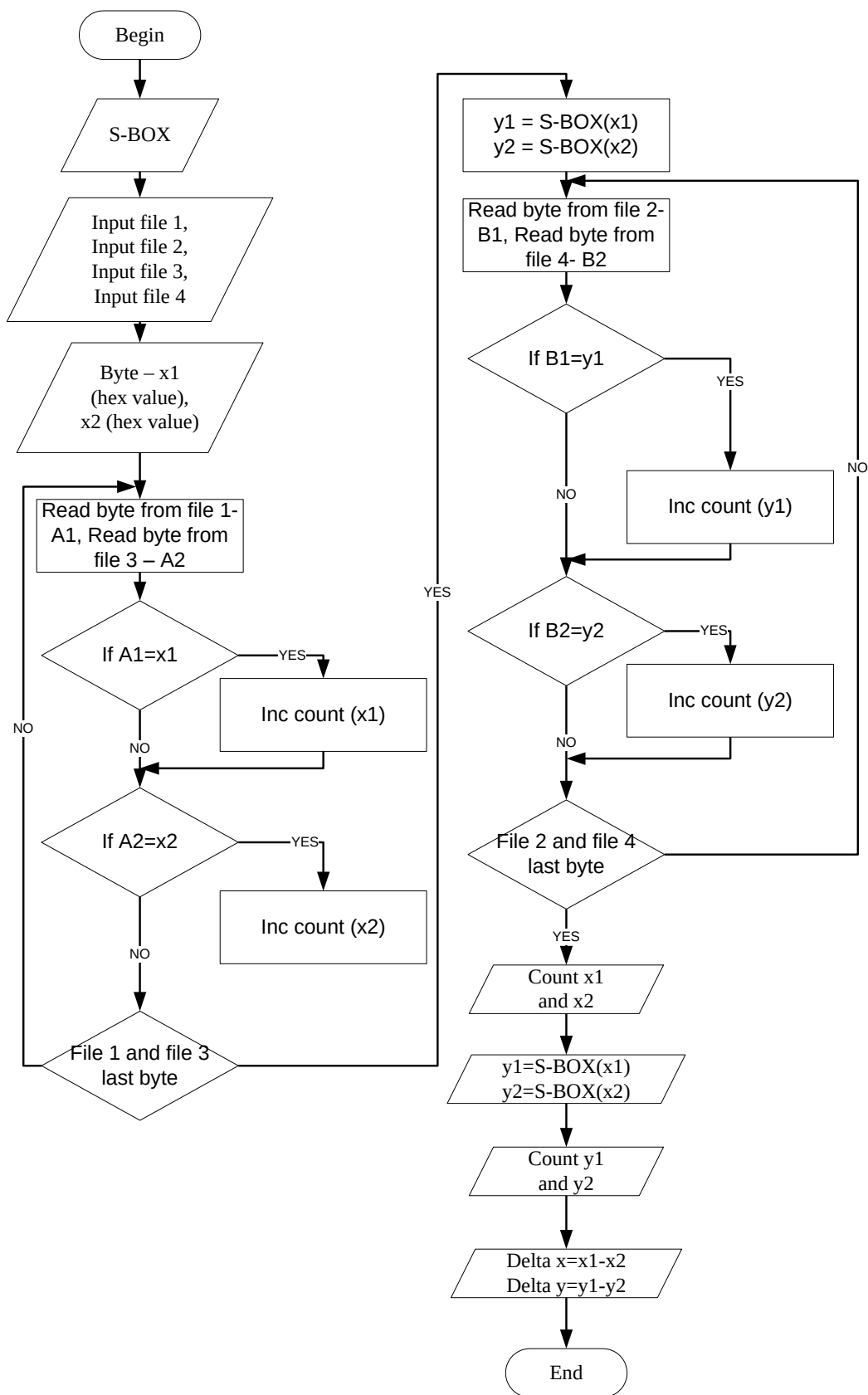


Figure 7.12: Dynamic I/O function scheme

- **Function “Dynamic O/I”**

The function „Dynamic O/I” checks the dynamic dependence (independence) between output and input data for a given s-box. The input data for this function is an inverted s-box, received from the checked one, the files (source 1 and source 2), files, received after processing with the function “File to S-BOX” with the checked s-box and the same input original files, bytes for which the dynamic input/output dependence will be checked. The selected invert s-box is loaded into a 16x16 matrix. The Binary File (destination 1) is read from the beginning to the end, counting the number of appearances of the value (y1). Based on the inverted s-box via the methods, described in the previous functions the original value of the checked (y1) introduced in Hexadecimal format (x1). The original file (source 1) is read from the beginning to the end, counting for the number of bytes with the given Hexadecimal value (x1). The described actions are repeated for the input file (source 2), output file (destination 2) and byte (x2).

The result of this function is: the number of appearances of the given Hexadecimal value (x1) in the Binary file (source 1), the inverted value of the checked byte y1, the number of appearances of the inverted byte in the Binary file (destination 1), the number of appearances of the given Hexadecimal value (x2) in the Binary file (source 2), inverted value of the checked byte y2, the number of appearances of the inverted byte in the Binary file (destination 2). The differences ($\Delta x = x1 - x2$ and $\Delta y = y1 - y2$) are calculated.

The results for the number of appearances and deltas have to be equal. On the Figure 7.13 the block-scheme of the algorithm of the “Dynamic O/I” function is shown.

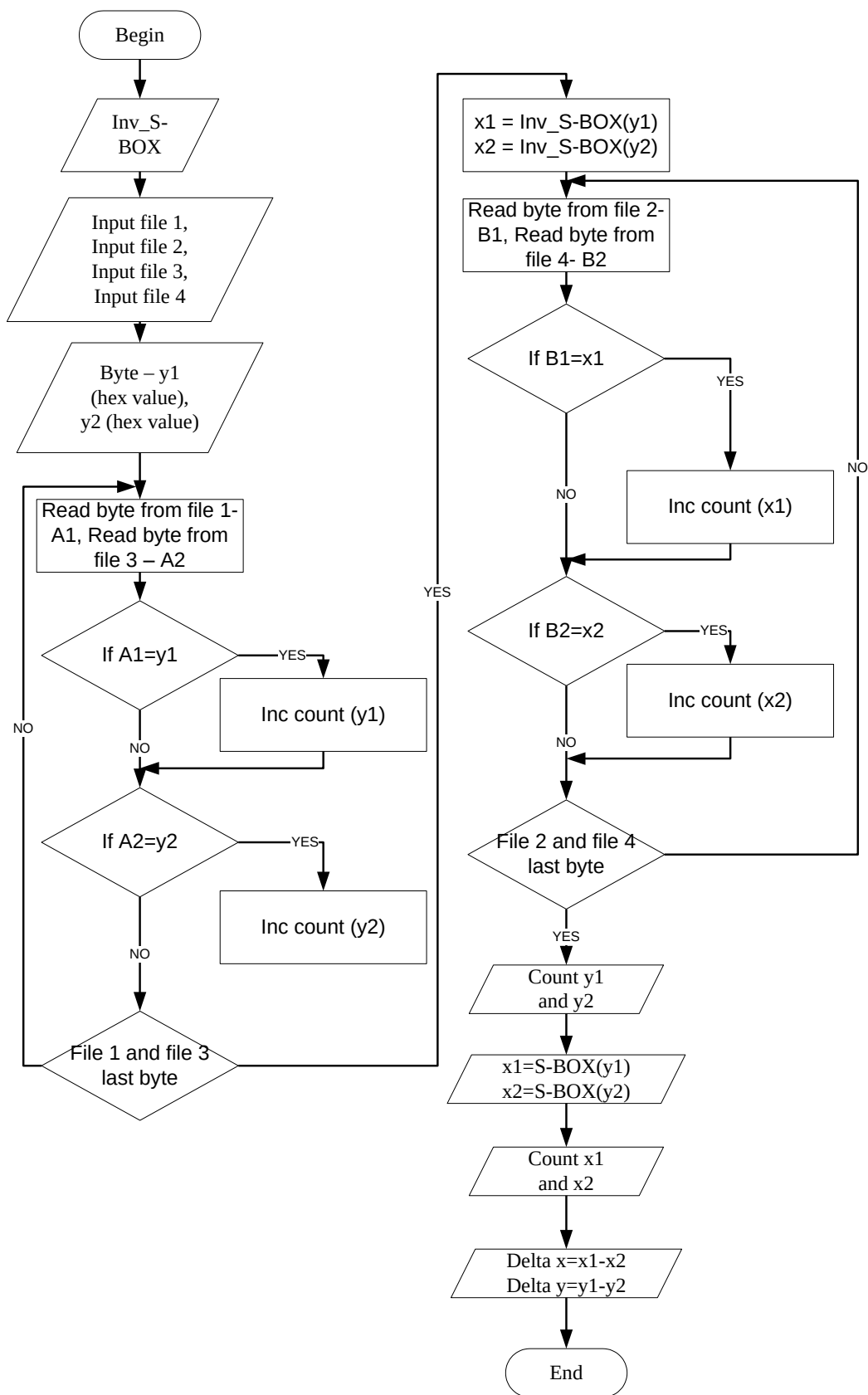


Figure 7.13: Dynamic O/I function scheme

8 Initial tests of the new cipher

Initial tests of the new cipher were carried out by means of the INDECT Block Cipher (IBC) application where the new block cipher was implemented (this application is described in Deliverable 9.13), plus the symmetric cipher simulator. Functions implemented by the simulator check some important properties of symmetric ciphers which help to evaluate cipher's security level.

For the purpose of our tests we took advantage of simulator functionality enabling checking AES-like S-boxes properties such as:

- Balancing
- Nonlinearity
- Strict Avalanche Criterion (SAC)
- Completeness
- Diffusion order
- XOR table distribution

Firstly, a matrix (S-box) representing a cipher algorithm was constructed in the following way. We put 256 zero bytes at the input of the encryption algorithm in CBC mode and obtained 256 output bytes. We used position of each byte as an index of the S-box table and the value of each byte as a proper S-box value. Obtained S-box representing encryption algorithm is presented in Listing 8.1. The initial vectors in the listings are plaintext and S-boxes represent ciphertext.

Listing 8.1: S-box representing the encryption algorithm (with key).

Initial vector:

```
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
```

Key:

```
100110101111010001100100110100010001010111101010010110011001101
0110101101100011100001110100000110110000010100100010101111001011
```

S-box:

```
50 56 A4 17 E0 76 B9 BB CE 85 F3 09 4E C6 42 2A
C6 A3 C2 28 E2 6E AE FA DF E2 9A 82 5B 93 19 99
00 90 3A 40 16 19 EB 11 1E F9 1B CF B5 80 2E 77
E7 4C AB BC 54 01 9B 45 B0 1D E8 FD B8 BD C4 5B
72 12 D8 EE B0 C5 6F 89 A9 35 69 B9 59 8C 13 AC
9B 73 BA A3 9D 79 37 10 05 C5 16 A1 08 D4 6E 8B
53 AE C6 49 9D 01 CA 46 A3 6B CC 2B 99 EA 8B 6D
8E A3 9B 92 B1 7B 67 F4 17 B8 7E 1F 83 EA AE 28
D2 85 D3 19 35 AE AC 34 26 E8 AD 82 4A C7 FC 93
B1 32 3D 6C CB 1D 87 16 DF 70 2A 67 57 7E 08 BE
9A 7C 10 20 FB 8F 5B A2 5A 3A 33 2D FF 3C F4 77
9B F3 4C 54 A0 37 37 A1 83 73 EF B5 39 9D D9 5A
85 D2 0E 7D 68 3E C9 D2 BE 7B 12 88 A7 54 58 31
F5 91 3A 6B F5 CB C4 66 44 CB 92 53 A0 48 58 C8
```

```
14 B6 F2 5B 8C 62 F1 70 97 26 C9 6B B2 AA 25 78
15 64 4E 4B B1 B3 77 00 8E B0 EF 8D 8A D4 F8 AA
```

In the second step, we examined these S-box properties by means of the symmetric cipher simulator. We present the results in Listing 8.2.

Listing 8.2: Results of checking S-box properties.

```
Probability of change output bit if one input bit has changed(SAC)
equals: 50%
Completeness is ensured for 100% possible inputs
faulty XOR distribution
diffusion order equals: 0
function is not balanced
minimum distance to affine function equals (bent function has 128):
for 1 bit 105      (most significant)
for 2 bit 92
for 3 bit 101
for 4 bit 105
for 5 bit 99
for 6 bit 104
for 7 bit 106
for 8 bit 106
```

To compare, we present the results of the simulation for S-box, which represents AES cipher in listing 8.3. This S-box was generated the same as in previous case.

Listing 8.3: S-box representing AES encryption algorithm and results of simulation

```
Initial vector:
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
```

```
Key (in hex):
01010101010001010100101010010101
```

```
S-box:
16 D3 B0 20 0B 12 05 39 60 3D 37 10 29 09 0F 43
B5 1C 38 E0 29 89 60 52 C2 06 43 D0 C0 4F DE B9
C2 20 65 44 B6 E1 44 DA 55 DA 12 20 98 31 22 50
CC 66 4E CC 22 04 4F 3F 03 0B E6 1E 68 6C B8 97
EF 56 DE 8A 54 A7 94 27 2B 2B 96 D0 B4 79 A2 28
6C 8B 7A B6 93 5D E7 3A 55 5C 30 EA 52 CF EF 13
BB 96 67 01 62 3E 95 77 E0 BB 2A 9A B6 C1 89 2A
52 E3 B7 BE 51 41 86 6E 69 51 A9 26 57 DD 82 9E
92 66 89 77 80 98 68 58 D0 5D 62 F7 DC A6 D2 AA
A7 FE 98 B1 79 A6 BB F2 0B 32 49 E8 E9 0F 91 C6
44 8D 88 B5 6A 0B 33 74 F4 E0 62 3E 64 FF D2 A3
66 81 AA 85 BF 00 74 26 FC 4A 09 42 B3 C6 1C 04
```



```

Probability of change output bit if one input bit has changed(SAC)
equals: 47%
Completeness is ensured for 100% possible inputs
faulty XOR distribution
diffusion order equals: 0
function is not balanced
minimum distance to affine function equals (bent function has 128):
for 1 bit 104      (most significant)
for 2 bit 102
for 3 bit 104
for 4 bit 104
for 5 bit 101
for 6 bit 98
for 7 bit 108
for 8 bit 96

```

Listing 8.5: S-box representing encryption algorithm and results of simulation

```
Initial vector:  
1010101010101010101010101010101010101010101010101010101010101010  
1010101010101010101010101010101010101010101010101010101010101010  
1010101010101010101010101010101010101010101010101010101010101010  
1010101010101010101010101010101010101010101010101010101010101010  
  
Key:  
0001001000001111000011011010110101100011110001000101011110100000  
1010111011110110100010010101001001111110001101001100010100000011
```

S-box:															
21	4F	EB	F1	10	47	F6	0E	F7	AE	09	DF	27	F7	1A	1D
E6	B4	EA	FB	A2	5B	82	F8	83	B6	F2	70	FF	30	B1	6F
A1	02	A7	4E	A0	FA	ED	0F	F9	A1	3E	64	04	43	C3	8F
7A	E0	D2	E9	73	1A	A2	5E	85	89	96	83	89	BE	36	8C
F7	71	B9	53	6F	DA	11	11	20	0F	A0	D5	D9	3A	43	02
9D	61	7E	EC	04	92	21	12	AA	06	BE	39	EB	D0	94	7D
67	A8	E9	EE	20	1C	12	28	EB	8C	F3	C1	EF	3C	38	1B
E8	26	D7	F6	62	EE	89	65	22	9D	C1	D1	3F	1E	E4	D0
00	CE	70	74	E3	FF	FB	4E	56	14	41	EE	80	AA	40	19
10	79	E4	71	44	B0	67	42	79	E4	1A	7E	D3	76	8B	95
D0	E6	41	93	BD	98	68	0D	C4	04	CB	53	B1	59	73	2C
81	71	32	1C	47	0F	2B	9D	7C	09	8C	95	D3	84	9D	A6
2D	F7	0F	01	60	BC	66	6A	D0	08	21	DC	B0	E0	22	60
AF	13	58	8A	D2	C8	78	08	6A	15	30	81	69	A2	08	5A
0B	33	E7	BD	72	B7	E8	E6	B5	3C	A5	E3	4C	0D	F6	32
2E	5F	F3	D6	9B	27	5F	FA	09	9E	59	18	FB	2D	D8	80

Probability of change output bit if one input bit has changed(SAC)
equals: 51%

Completeness is ensured for 100% possible inputs
faulty XOR distribution

```
diffusion order equals: 0
function is not balanced
minimum distance to affine function equals (bent function has 128):
for 1 bit 105      (most significant)
for 2 bit 100
for 3 bit 106
for 4 bit 107
for 5 bit 103
for 6 bit 103
for 7 bit 106
for 8 bit 104
```

In the Listing 8.6 the letters of alphabet are employed in order to make the initial vector.

Listing 8.6: S-box representing encryption algorithm and results of simulation

Initial vector:
 abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklm
 nopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklm
 nopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklm
 nopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklm
 nopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklm

Key:
 1111001100001111101100011011001000100110101101000101010110110010
 0110000011001000100110011011101110100110010100110111101001110110

S-box:

F3	47	EC	94	7C	A1	44	1F	4A	73	9F	01	07	8C	8F	D3
1C	08	E1	1B	1E	99	96	A0	CE	C1	AE	AE	D8	F1	B0	FE
09	2F	6B	34	60	61	54	E3	B9	FF	AC	02	0E	51	0F	50
96	FC	91	5A	76	1F	B0	AA	0A	6F	BE	81	DA	F9	F9	22
CF	3D	D0	6A	CD	5C	7F	E6	05	C4	D2	A3	B4	20	3A	70
3E	D4	D2	13	9B	10	66	39	33	09	C6	D9	F4	79	91	1C
B2	13	2B	A6	AE	DF	AB	FD	FC	F0	0D	85	40	84	5A	8B
35	E7	83	40	96	C1	89	86	38	1E	CB	CA	27	5A	85	1B
68	23	ED	50	F4	17	A0	2E	09	92	C4	F9	99	45	37	83
89	81	CD	13	9A	97	34	D5	44	7A	D7	60	49	5A	58	F5
2D	DA	AE	53	3C	09	18	D1	9B	1A	9C	88	52	BC	92	4D
5E	EC	BE	B5	3A	F7	04	A9	70	EA	12	5B	EB	B9	41	B5
0A	DC	08	E2	64	15	C9	33	24	2C	6E	C1	2B	DD	4F	E4
A4	20	AE	52	2F	67	34	F6	95	86	74	7F	57	C1	CF	7D
7B	31	71	15	10	B8	7D	A6	CC	E0	B1	4E	26	A3	8F	EE
61	2E	0D	E0	89	4A	EB	3B	16	67	91	8E	1F	C7	EB	0A

```
Probability of change output bit if one input bit has changed(SAC)
equals: 50%
Completeness is ensured for 100% possible inputs
faulty XOR distribution
diffusion order equals: 0
function is not balanced
minimum distance to affine function equals (bent function has 128):
for 1 bit 106      (most significant)
for 2 bit 106
for 3 bit 104
```

```
for 4 bit 103
for 5 bit 106
for 6 bit 104
for 7 bit 102
for 8 bit 104
```

In the case presented by Listing 8.6, there are very good S-box parameters (especially high nonlinearity). In the Listing 8.7 we encrypted the natural numbers (beginning from number 1).

Listing 8.7: S-box representing encryption algorithm and results of simulation

Initial vector:

```
1234567891011121314151617181920212223242526272829303132333435363
7383940414243444546474849505152535455565758596061626364656667686
9707172737475767778798081828384858687888990919293949596979899100
1011021031041051061071081091101111121131141151161171181191201211
```

Key:

```
1100100001010010100001111111111010011001110101010111111101000000
0100000110010100100001010001101101001101010111010011101011101000
```

S-box:

```
F9 8B E2 FD E5 48 88 14 04 0E A6 8D 1A 7F 4F F0
E9 77 66 D3 77 60 E7 31 5D DE 7A AA 52 09 C7 28
C8 0E 64 9F 2B CC 23 67 F8 FC 56 51 CB 04 43 CD
C8 3F AC 89 38 FC 75 91 91 71 F4 16 92 C9 1F D1
BE 90 F6 88 79 46 93 4D 63 5B 0B F2 85 AB EC 2D
23 E2 57 14 43 2E F3 2D 10 D8 BE E4 5F 79 CF 0B
EF AB DA 37 A2 9C 28 45 A7 42 21 9C 8C A1 83 8E
5D 7F 90 8F 44 E2 8A 7D 3E 5D CC 54 4A 61 FF 02
74 BE 1C 50 A3 B0 60 47 65 77 E7 AF E3 9B 02 08
8E F0 8F 5F 1A 12 71 45 7C 43 81 5D 12 BA 0E BE
8D E0 9C 79 AC 7B 1A 18 32 C7 79 D7 81 79 C3 8B
97 67 FA 78 1F 76 5D B0 C8 A7 CC 1A 37 D7 A7 84
22 83 0F 41 73 D1 92 87 D1 8C 5C 52 DC 76 CF 74
5C FD 74 5A FB 7D DC A1 39 F3 C8 22 31 2D 1D F9
06 22 DA DB 29 6C 08 C0 7C 10 6A 9C BE ED 8E 76
BC C9 4B 45 F0 92 94 3D 53 42 D6 B3 0D 5D BF CB
```

Probability of change output bit if one input bit has changed(SAC)
equals: 49%

Completeness is ensured for 100% possible inputs
faulty XOR distribution

diffusion order equals: 0

function is not balanced

minimum distance to affine function equals (bent function has 128):

for 1 bit 106 (most significant)

for 2 bit 96

for 3 bit 104

for 4 bit 104

for 5 bit 103

for 6 bit 100

for 7 bit 102

for 8 bit 102

In the Listing 8.8 random generated characters make the initial vector.

Listing 8.8: S-box representing encryption algorithm and results of simulation

Initial vector:

```
:kqgh\dir]zq$^"~^£1-%.uv5%es!nh1[qzc(312*z07*\tu+*3£0v@[b+o"5fc
kn]4-!xm+$6r#!\j~[b36_~^£v1^blmlm3z2r?%f5(d\*dd~*a^7ip#nk7£\<£a7
n_($hu`dq^.\>o,0f0i-[^wem;4s)ogf_/'^=^`]b.4i+£x6,ach&9xgyv+="slt^
6n?<2;&3,4hey;);5i!hz&fnd6[i|b(0o1/,?#hyq#v'>y:%!8w|b$amn@(kf0$h
```

Key:

```
0001111000000110101100100110111001010010011010101101011000011001
010010001011011001100010011111010000110111110010000001000110001
```

S-box:

30	B4	34	4D	4B	01	CD	4E	42	92	33	0D	69	D2	6B	9A
6B	88	E9	C6	48	76	0A	52	80	FD	43	E5	0F	DD	DE	93
F9	25	F9	DE	CD	D2	E4	9C	5B	77	5F	8A	54	5F	BD	8D
F0	91	13	64	C9	29	37	AF	73	DE	B5	F1	96	25	09	BF
21	07	C1	0B	42	5A	88	B1	84	43	B9	25	C1	F4	9A	00
98	25	33	0B	05	46	F7	6D	37	A4	AB	44	AC	67	33	23
AB	46	AB	4F	A5	60	EC	63	AA	A7	1E	76	BB	29	36	9E
4A	FC	A0	BA	B4	A6	13	0D	F7	10	EC	87	25	02	2A	70
32	F3	B9	3B	D3	C0	4E	21	2D	57	9F	7C	06	FF	E4	A1
76	10	A9	BB	7C	72	81	B3	B6	F9	72	D4	C7	4B	D1	64
92	91	C5	C3	8F	24	1A	48	70	E6	FE	76	C2	15	A7	1B
93	A8	64	77	E4	AB	C9	C5	A6	63	FD	95	65	F8	15	DC
2D	94	65	27	9F	D3	10	F2	E4	25	97	A7	26	4F	8C	DA
50	A1	EB	23	8A	30	41	54	99	0E	6F	38	5B	52	6B	47
27	93	21	56	29	48	73	C4	4B	6C	16	95	4F	EC	64	0E
6E	AF	34	3F	81	FB	0D	27	07	4B	0B	F0	16	2F	C5	B7

Probability of change output bit if one input bit has changed(SAC)
equals: 49%

Completeness is ensured for 100% possible inputs

faulty XOR distribution

diffusion order equals: 0

function is not balanced

minimum distance to affine function equals (bent function has 128):

for 1 bit 102 (most significant)

for 2 bit 103

for 3 bit 104

for 4 bit 106

for 5 bit 108

for 6 bit 104

for 7 bit 101

for 8 bit 96

9 Ethical issues and security considerations

The information in this paper is meant for public use. The proposed block cipher has been designed to ensure data confidentiality, which is one of the basic security requirements [6]. Therefore, the whole document concerns security issues, and all sections of this deliverable require full awareness of security.

The objective of this chapter is to present a new block cipher. This cipher is an encryption/decryption algorithm that is able to protect sensitive data against unauthorized access. It is thus able to ensure confidentiality as well as the privacy of data. However, this deliverable does not raise any ethical issues.

10 Conclusions

This deliverable presents a novel idea to create innovative block ciphers. The new algorithm consists of only nonlinear transformations, which are dependent on key. This feature ensures a high level of security. Additionally, the large number of different S-boxes makes each encryption highly unique. We have performed a lot of tests and simulations to confirm the high level of security. Also, we have also carried out the cryptanalysis of new block cipher to evaluate its security and possible vulnerabilities. The construction of this cipher (substitution and permutation functions in repetitive rounds) also ensures a good performance and fast encryption/decryption process.

The idea and assumptions of new block cipher were received with interest by potential end-users. Nevertheless, currently it could be used to encrypt only non-classified information. Encryption of classified information could be possible only after the special certification process.

In the future we are going to develop the new cipher and perform additional security tests. Besides the improvement of the cipher, we want to implement some methods of concurrent error detection. The results of this research will be presented in the deliverable D8.5 “Definition of new methods of concurrent error detection”. Also we are going to implement the new block cipher in hardware—with use of a field-programmable gate array (FPGA) circuits.

Abbreviations

AES	Advanced Encryption Standard
CBC	Cipher Block Chaining
DES	Data Encryption Standard
ECB	Electronic CodeBook
FPGA	Field Programmable Gate Array
IDEA	International Data Encryption Algorithm
I/O	Input/Output
O/I	Output/Input
SAC	Strict Avalanche Criterion
S-Box	Substitution Box
XOR	eXclusive OR

References

- [1] Douglas R. Stinson, “*Cryptography: theory and practice* 3rd ed. (Discrete mathematics and its applications)”, 2002
- [2] M. Wenbo, “*Modern Cryptography: Theory and Practice*”. Prentice Hall PTR, 2003.
- [3] *Announcing the ADVANCED ENCRYPTION STANDARD (AES)*, Federal Information Processing Standards Publication 197, November 2001
- [4] Howard M. Heys, *A Tutorial on Linear and Differential Cryptanalysis*, Electrical and Computer Engineering Faculty of Engineering and Applied Science Memorial University of Newfoundland St. John’s, NF, Canada
- [5] Diploma Thesis of Pascal Junod, “*Linear Cryptanalysis of DES*” ETZ Zurich
- [6] ITU-T X.805 “*Security architecture for systems providing end-to-end communications*”. 2003.

Document Updates

Version	Date	Updates and Revision History	Author
v0.1	10/09/2010	Initial Table of Contents for discussion	Marcin Niemiec
v0.2	22/09/2010	Table of Contents	Marcin Niemiec et al.
v0.3	12/10/2010	First inputs	Marcin Niemiec et al.
v0.4	20/10/2010	First Draft	Marcin Niemiec
v0.5	05/10/2010	Final inputs	Marcin Niemiec et al.
v0.6	10/11/2010	Complete draft	Marcin Niemiec et al.
v1.0	29/11/2010	First complete version for review	Marcin Niemiec
V1.1	08/12/2010	Complete document	Marcin Niemiec